

UNIVERSIDAD DE EL SALVADOR

Facultad Multidisciplinaria De Occidente
Departamento de Ingeniería y Arquitectura
Ingeniería en Desarrollo de Software



Asignatura:

Introducción al Software Libre

Ciclo IV/ Segundo año

Tema:
Proyecto Final

Tutor/a GT01:
Ing. Francisco Javier Morales Ayala

Alumno/a:
Grupo 5
Fernando Jose Tesorero Brito
Luis Ángel Huezo Martínez
Jorge Mario Meléndez Chicas
Steven Alexis Martinez Jimenez

Fecha de entrega:
03/11/2026

Introducción **3**

Servicios implementados 3

1. Preparación del entorno servidor **3**

1.1 Administración básica del sistema 3

1.2 Estructura de directorios y permisos 3

2. Automatización y monitoreo **4**

2.1 Script de monitoreo del sistema 4

2.2 Automatización con cron 4

3. Control de versiones **6**

4. Docker **8**

4.1 Instalación y configuración 8

4.2 Verificación inicial 10

5. Servidor web containerizado **12**

5.1 Contenedor Nginx básico 12

5.2 Verificación del servicio web 12

6. Docker avanzado **15**

6.1 Dockerfile personalizado 15

6.2 Contenedor con imagen personalizada 17

7. Redes y volúmenes Docker **20**

7.1 Red personalizada 20

7.2 Volúmenes persistentes 22

8. Pruebas y verificación final **27**

8.1 Pruebas de conectividad 27

8.2 Verificación integral 27

Introducción

Este informe documenta el Proyecto Final de Introducción al Software Libre (ISL135, Ciclo II 2026): la implementación de un servidor Linux automatizado con contenedores Docker, integrando administración de sistemas, control de versiones y virtualización. El trabajo corresponde al Grupo 5 y se realizó en una máquina virtual Ubuntu con el nombre de host servidor-grupo5.

Cada apartado incluye la explicación de la fase, los comandos ejecutados y las capturas de pantalla que sirven de evidencia. El código y los scripts están en el repositorio de GitHub proyecto-linux-grupo5.

Servicios implementados

Contenedor	Imagen	Puerto del host	Apartado
nginx-basico	nginx	8080	5
servidor-grupo5	servidor-grupo5 (imagen propia)	8081	6
nginx-red	nginx (red red-proyecto)	8082	7.1
nginx-volumen	nginx (volumen volumen-web)	8083	7.2

1. Preparación del entorno servidor

1.1 Administración básica del sistema

Se asignó al equipo el nombre servidor-grupo5 y se crearon los usuarios adminsys, tecnico (escrito sin tilde, porque los nombres de usuario en Linux deben evitar caracteres acentuados) y visitante. Se crearon los grupos soporte y web, y se asignaron así: adminsys con privilegios de sudo, tecnico en el grupo soporte y visitante en el grupo web. Todo el proceso está automatizado en el script 01_preparar_entorno.sh.

```
sudo bash /proyecto/scripts/01_preparar_entorno.sh 5
hostnamectl
id adminsys
id tecnico
id visitante
```

1.2 Estructura de directorios y permisos

Se creó en /proyecto/ la estructura datos/ (archivos de configuración), web/ (sitio web), scripts/ (automatización) y capturas/ (evidencias). El directorio datos/ pertenece al grupo soporte y web/ al grupo web. Ambos se configuraron con el bit setgid (permisos 2775) para que los archivos nuevos hereden el grupo del directorio.

```
ls -ld /proyecto /proyecto/*
```

2. Automatización y monitoreo

2.1 Script de monitoreo del sistema

El script reporte_sistema.sh, ubicado en /proyecto/scripts/ y con permisos de ejecución, muestra la fecha y hora actual, el nombre del host, el número de usuarios conectados, el espacio libre en el disco principal, la memoria RAM disponible y el número de contenedores Docker activos.

```
ls -l /proyecto/scripts/reporte_sistema.sh
/proyecto/scripts/reporte_sistema.sh
```

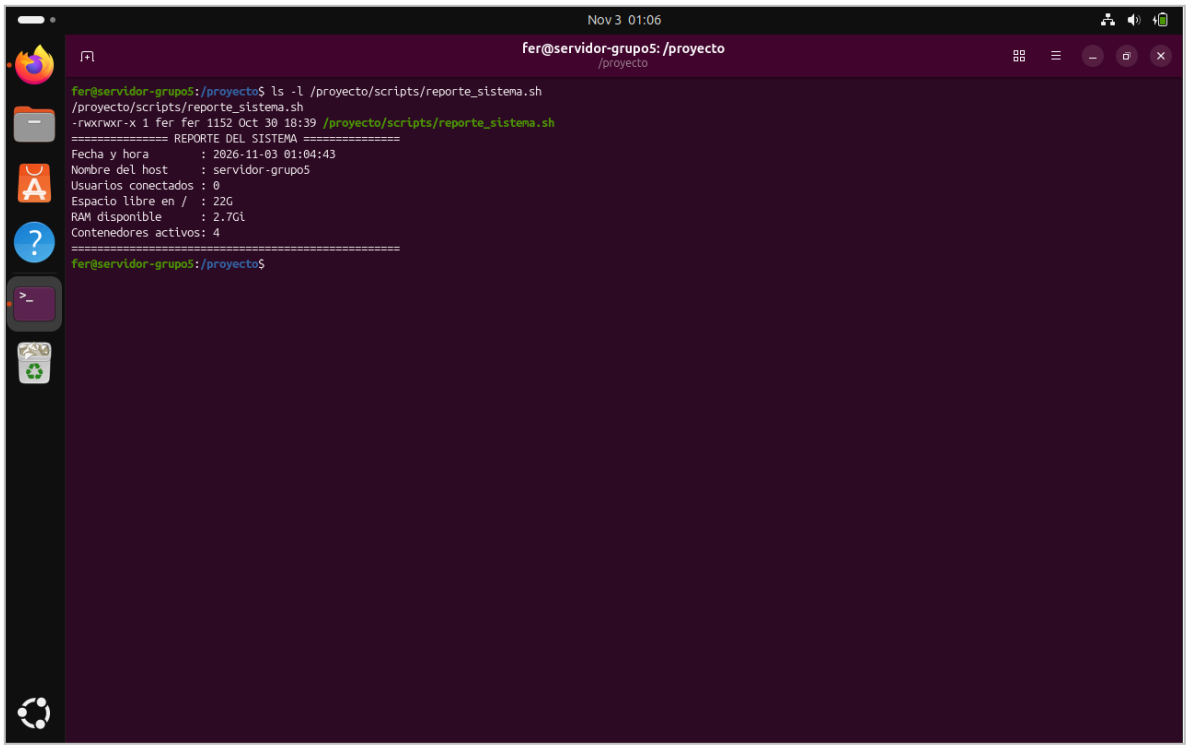


Figura 1. *Permisos de ejecución de reporte_sistema.sh y su salida con los seis datos solicitados.*

2.2 Automatización con cron

Se configuró una tarea de cron que ejecuta reporte_sistema.sh cada 30 minutos (*/30 * * * *) y guarda la salida en /var/log/proyecto/reporte_sistema.log. Se verificó que el servicio cron está activo, que la tarea existe en el crontab y que el registro se actualiza con nuevas entradas.

```
systemctl is-active cron
sudo crontab -l
tail -n 12 /var/log/proyecto/reporte_sistema.log
```

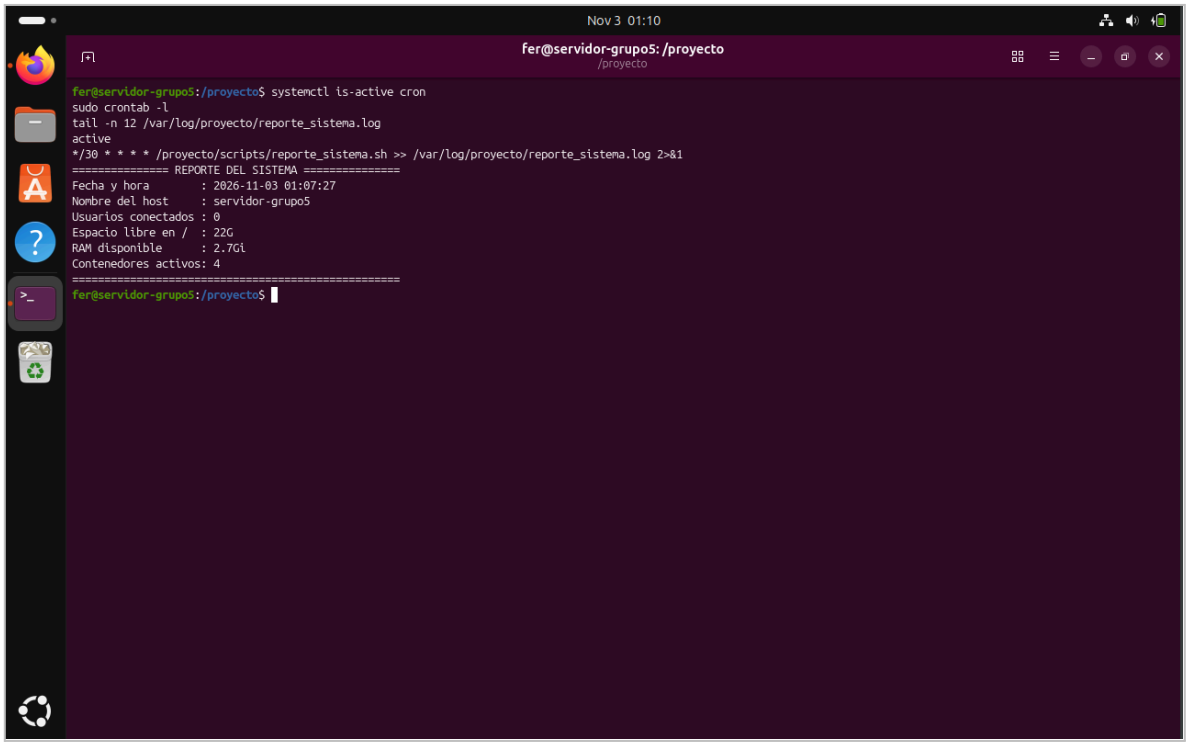


Figura 2. *Servicio cron activo, tarea programada cada 30 minutos en el crontab y últimas líneas del registro.*

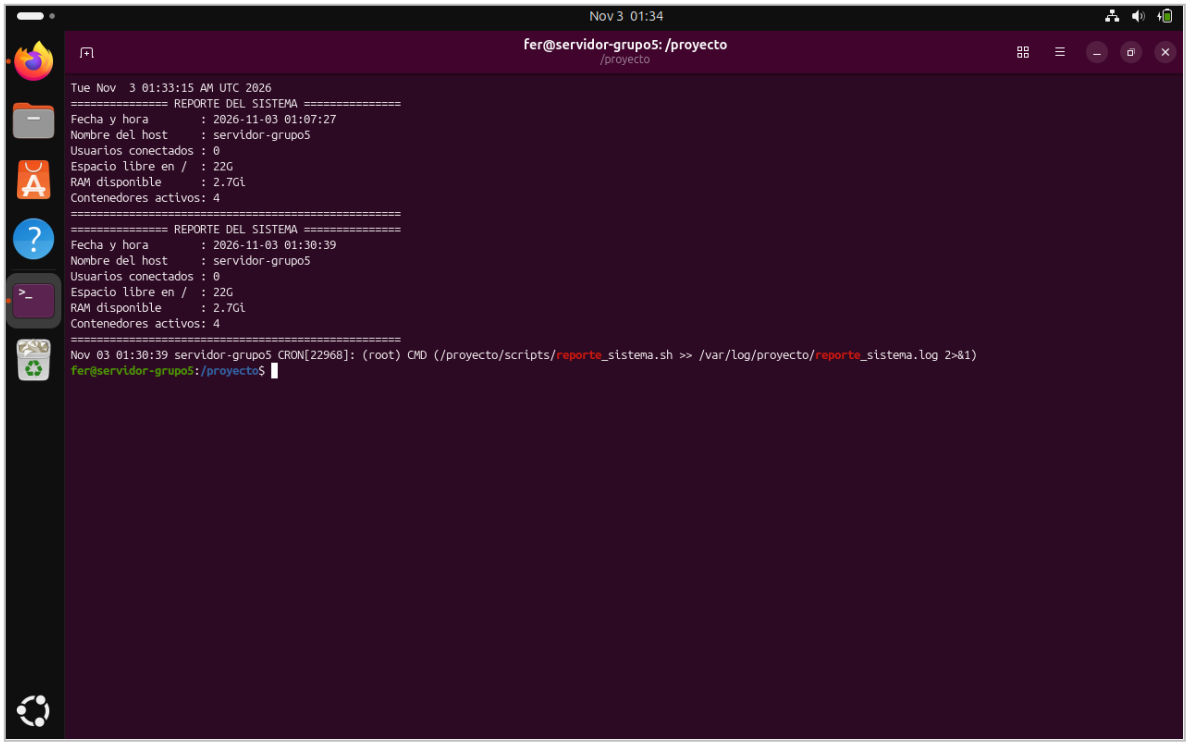
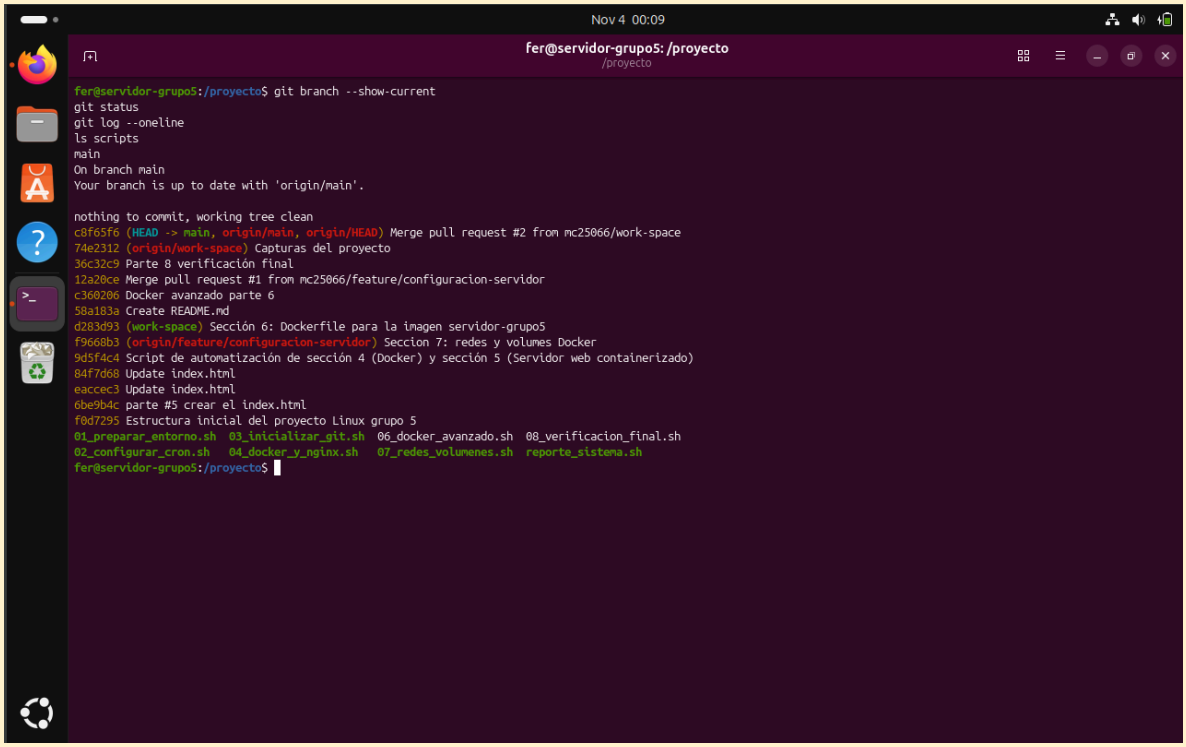


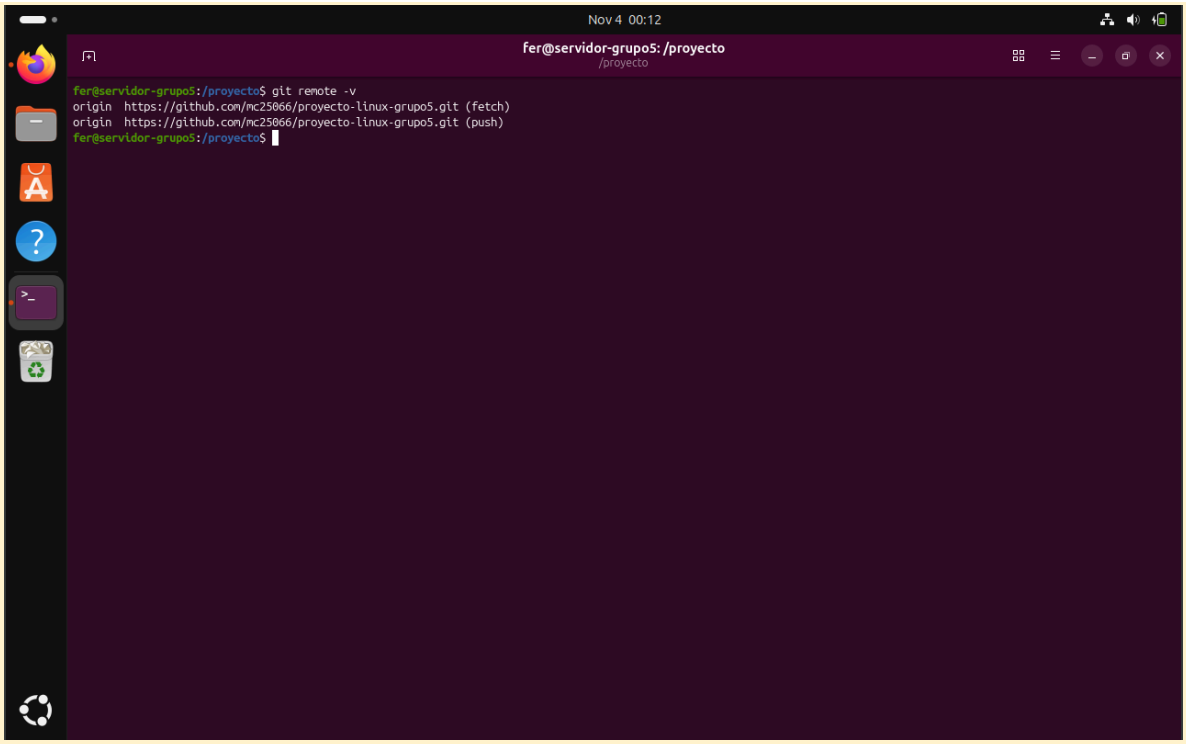
Figura 3. *El registro se actualiza con una nueva entrada y el sistema confirma que cron ejecutó el script (01:30).*

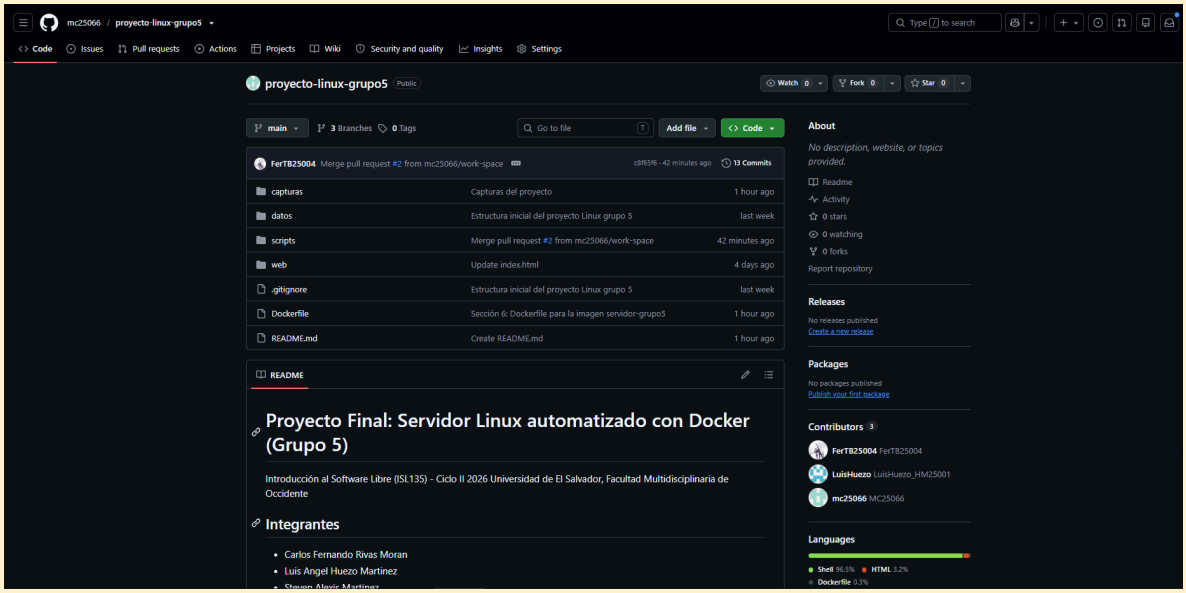
3. Control de versiones

Se inicializó un repositorio Git en /proyecto/, se realizó el primer commit con toda la estructura del proyecto y se creó en GitHub el repositorio proyecto-linux-grupo5, al que se subió el código local. El equipo trabaja en la rama work-space. Finalmente se agregó un archivo README.md con la descripción del proyecto y los pasos de instalación.

- git log --oneline y git remote -v en /proyecto.
- El repositorio proyecto-linux-grupo5 en GitHub con la estructura y los scripts.
- El archivo README.md.







[Repositorio de GitHub del equipo 5]

4. Docker

4.1 Instalación y configuración

Se instaló Docker desde los repositorios de Ubuntu (paquete docker.io), se habilitó e inició el servicio y se agregaron los usuarios adminsys y tecnico al grupo docker para que puedan ejecutar comandos Docker sin sudo. Se comprobó la instalación y que adminsys ejecuta docker ps sin sudo.

```
sudo apt update
sudo apt install -y docker.io
docker --version
sudo systemctl enable --now docker
systemctl is-active docker
sudo usermod -aG docker adminsys
sudo usermod -aG docker tecnico
id adminsys
id tecnico
su - adminsys
docker ps
```

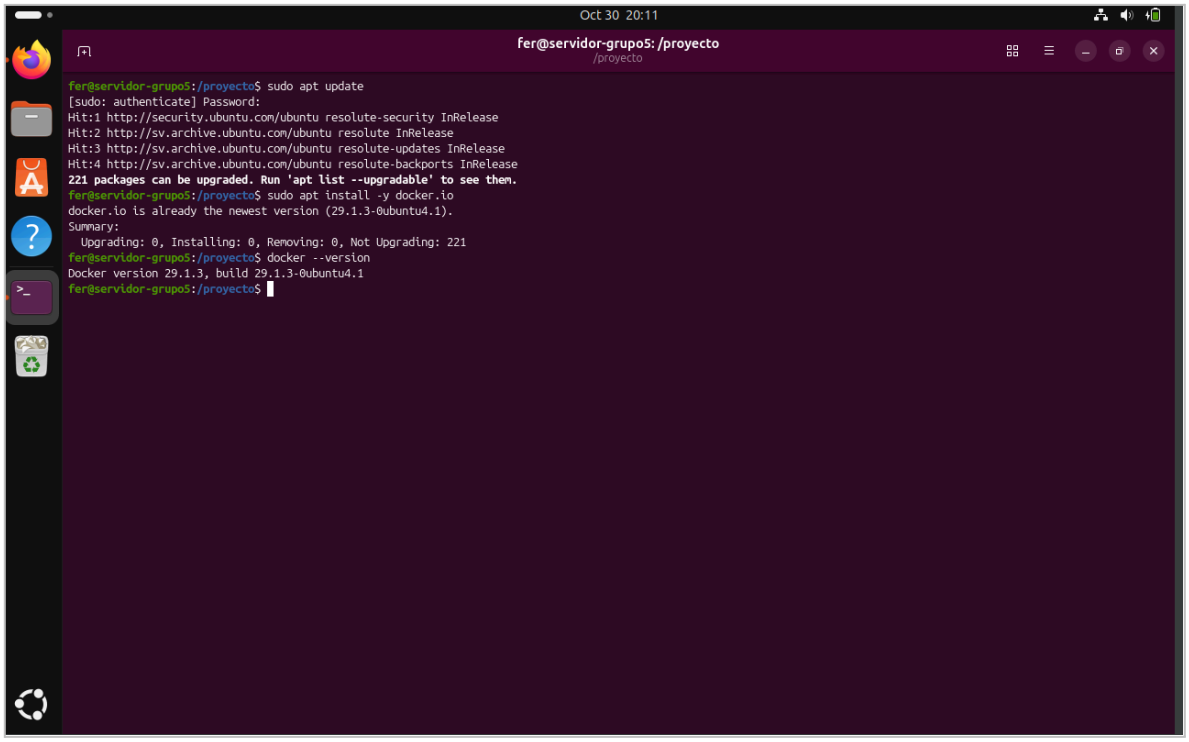


Figura 4. *Instalación de Docker (docker.io) y versión instalada.*

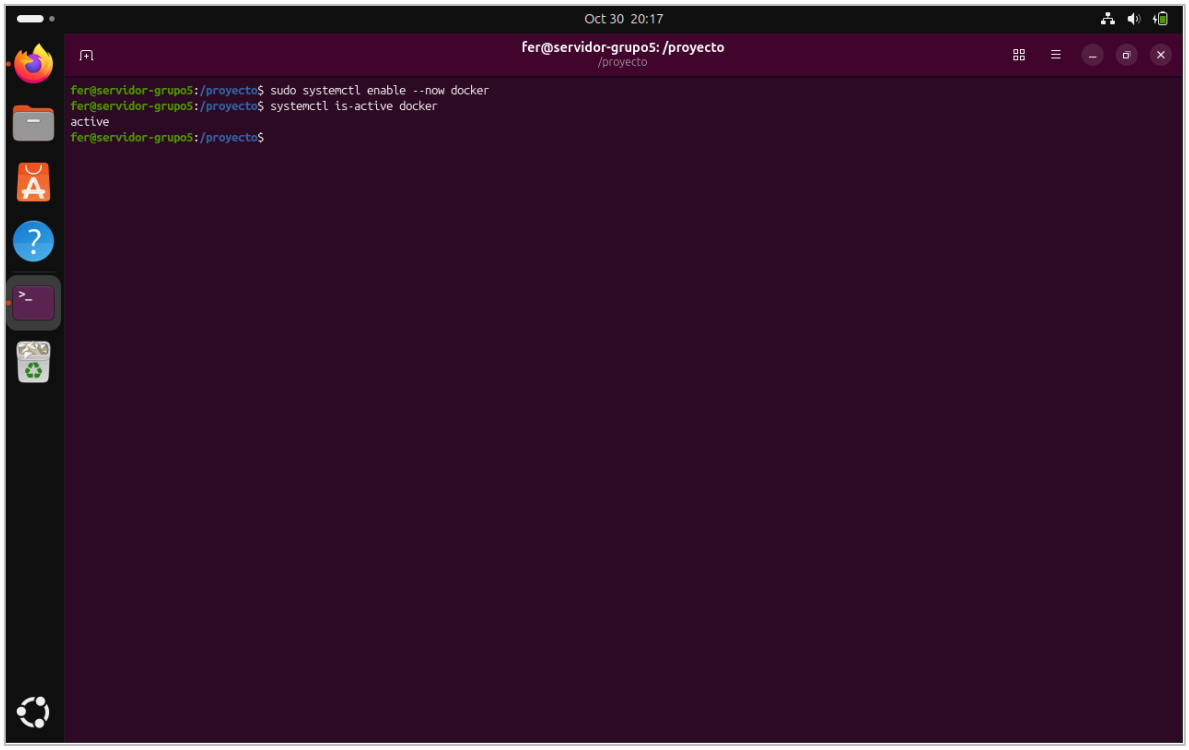


Figura 5. *Servicio Docker habilitado, iniciado y activo.*

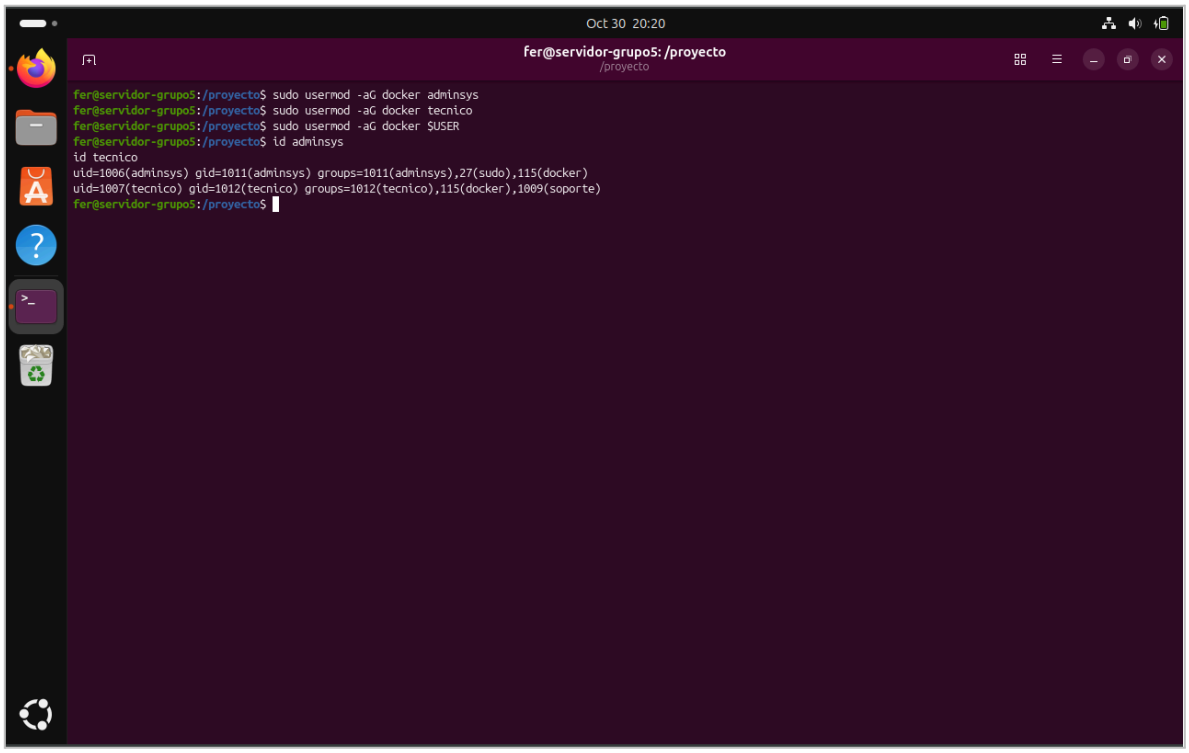


Figura 6. *Usuarios adminsys y tecnico agregados al grupo docker.*

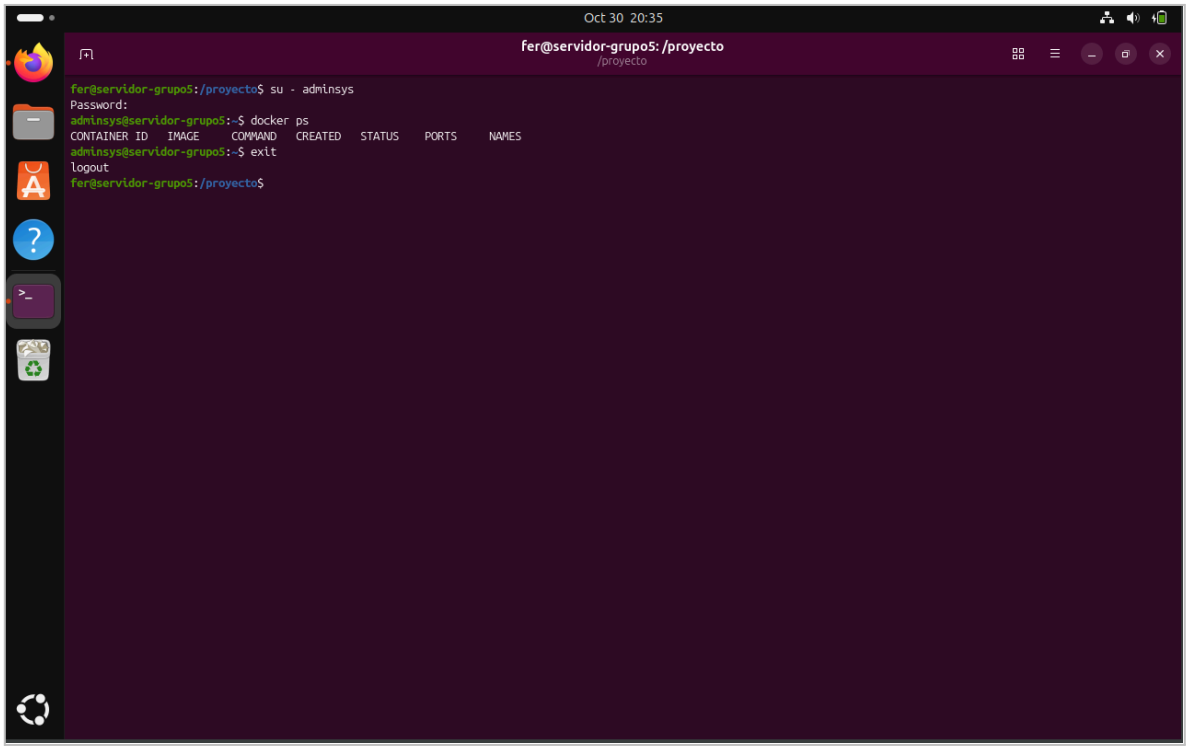


Figura 7. *adminsys ejecuta docker ps sin necesidad de sudo.*

4.2 Verificación inicial

Se ejecutó el contenedor de prueba hello-world para confirmar que Docker puede descargar imágenes y ejecutar contenedores, y se listaron todos los contenedores, donde hello-world aparece con estado Exited (0), que es lo esperado.

```
docker run hello-world
docker ps -a
```

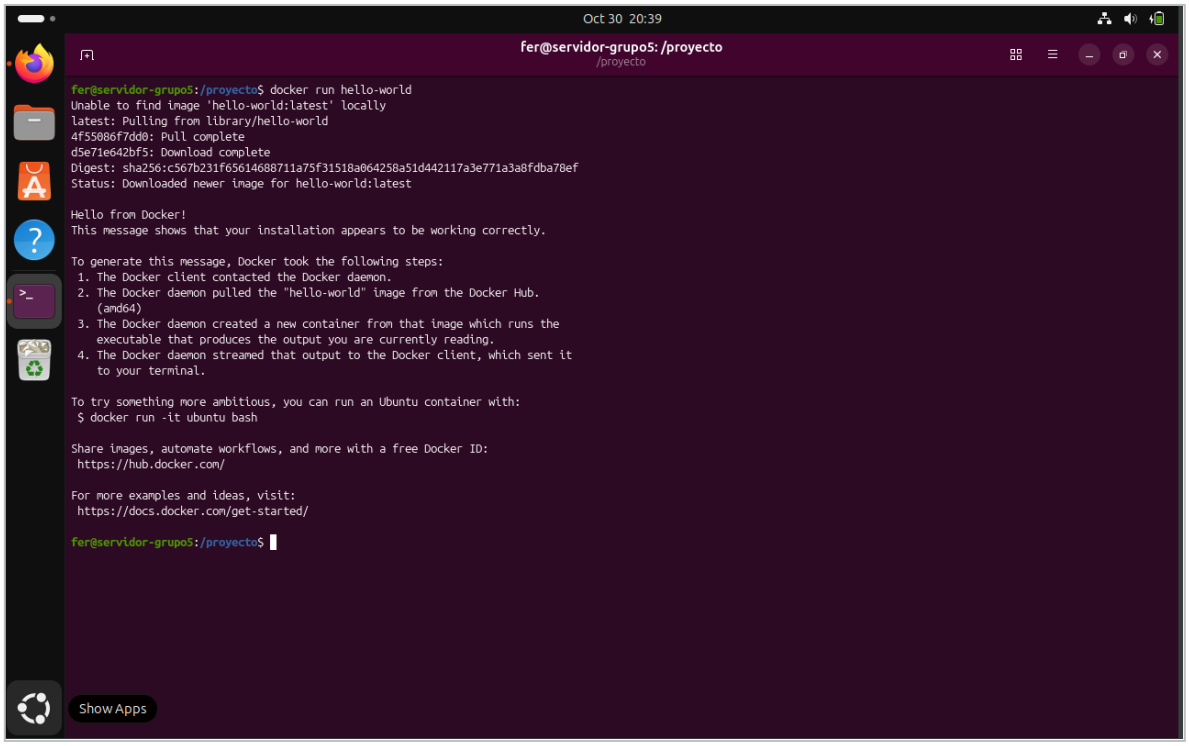


Figura 8. *Descarga y ejecución de hello-world con su mensaje de éxito.*

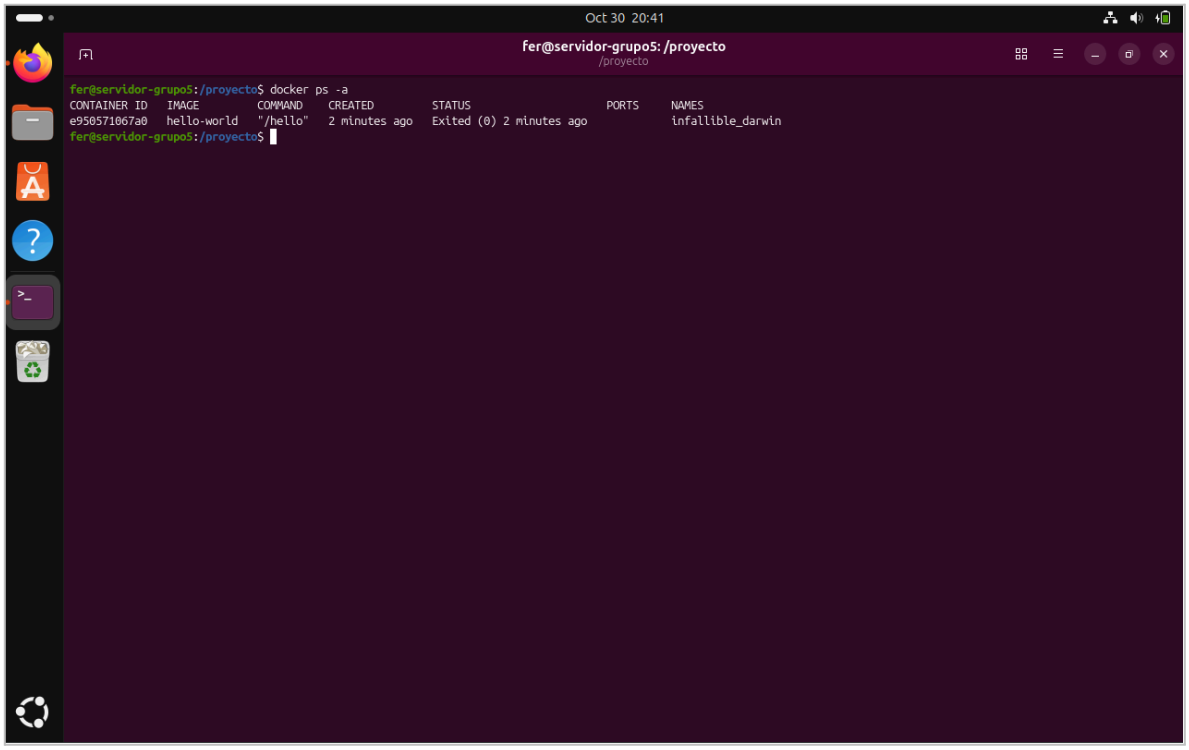


Figura 9. *docker ps -a: el contenedor hello-world finalizó con estado Exited (0).*

5. Servidor web containerizado

5.1 Contenedor Nginx básico

Se creó el archivo /proyecto/web/index.html con el título del proyecto y los integrantes del grupo, y se levantó un contenedor Nginx en segundo plano (detached) que mapea el puerto 8080 del host al 80 del contenedor y monta /proyecto/web en /usr/share/nginx/html.

```
nano /proyecto/web/index.html
chmod 644 /proyecto/web/index.html
docker run -d --name nginx-basico -p 8080:80 -v
/proyecto/web:/usr/share/nginx/html nginx
docker ps
```

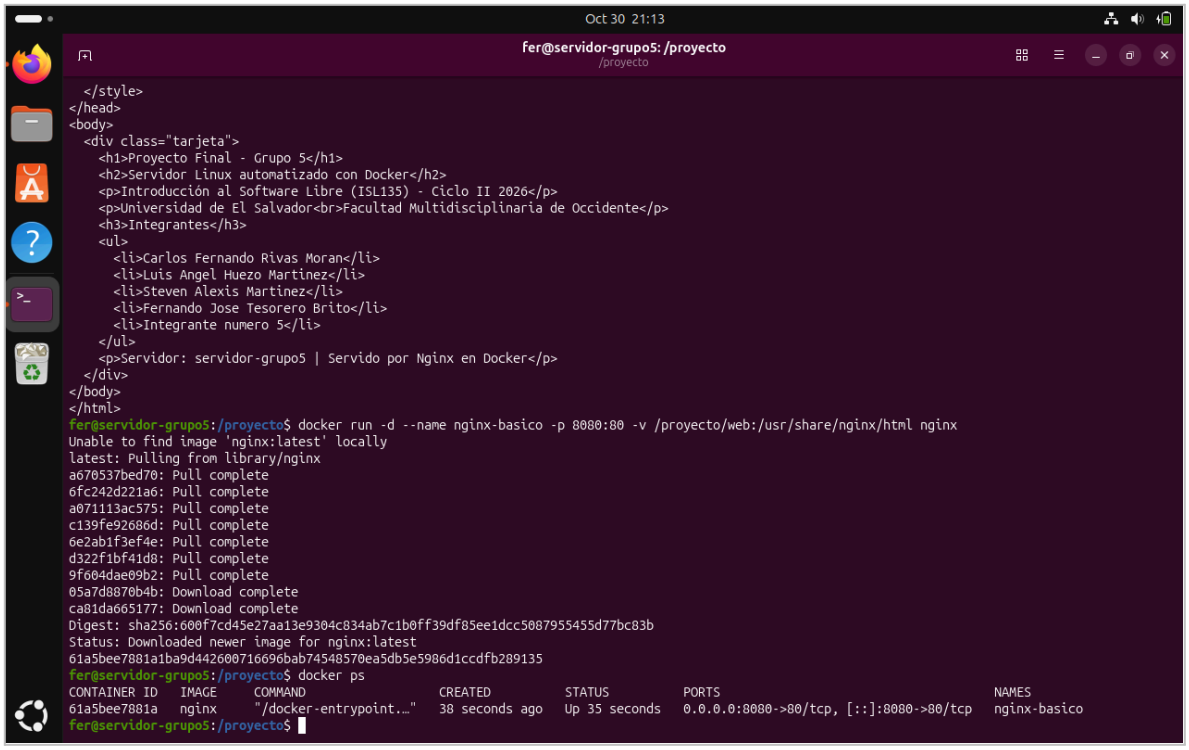


Figura 10. *Contenido de index.html y contenedor nginx-basico en ejecución (puerto 8080, directorio web montado).*

5.2 Verificación del servicio web

Se comprobó el servicio con curl, se revisaron los logs del contenedor (arranque de Nginx sin errores) y se abrió la página en el navegador en http://localhost:8080.

```
curl http://localhost:8080
docker logs nginx-basico
```

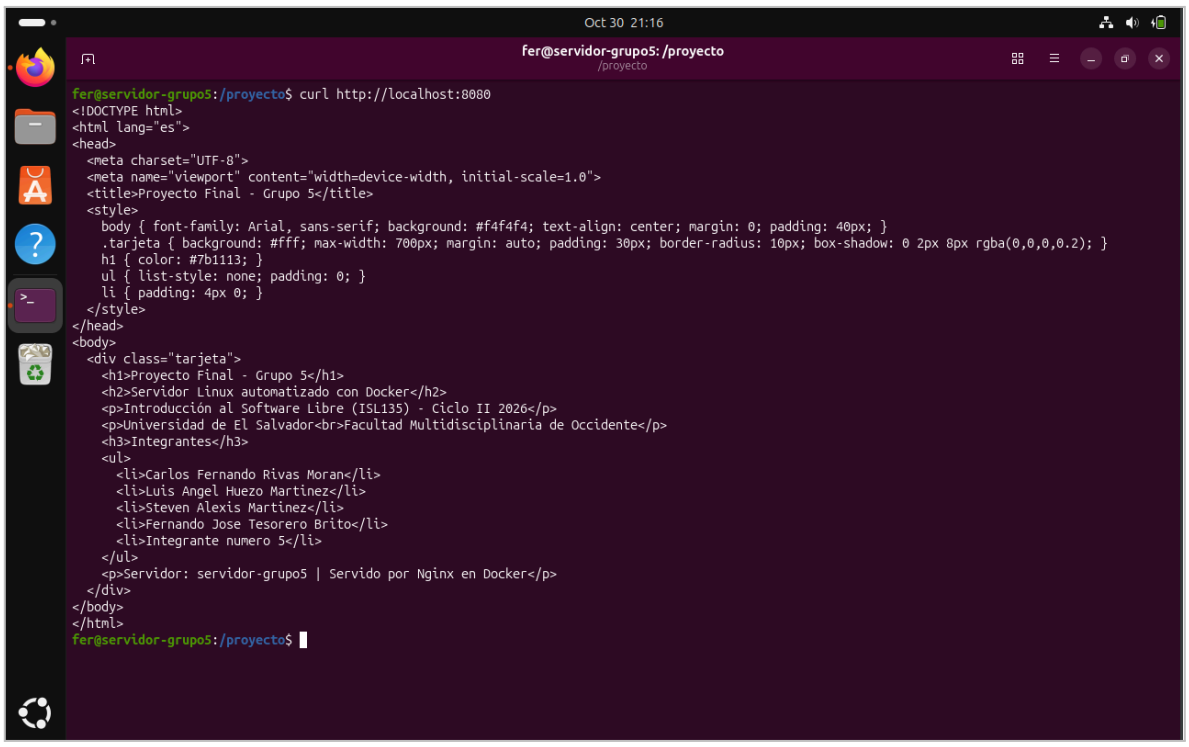


Figura 11. *curl a http://localhost:8080: el servidor responde con el index.html del grupo.*

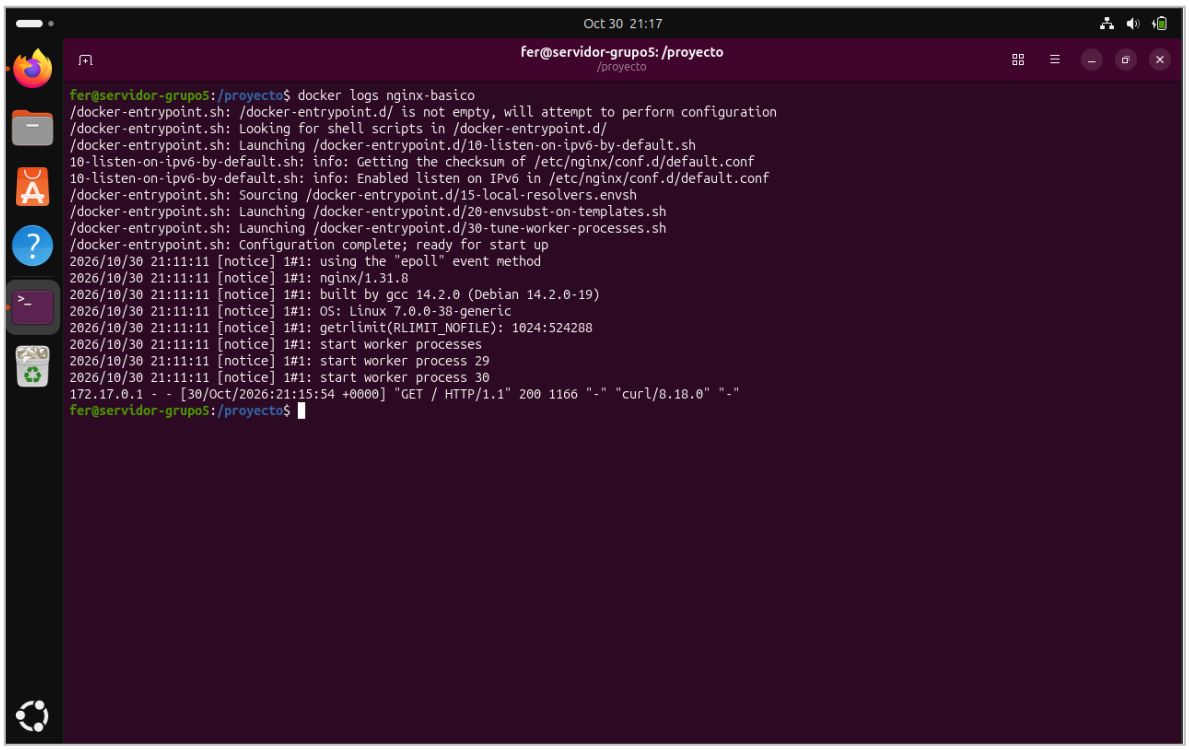


Figura 12. *Logs del contenedor nginx-basico sin errores.*

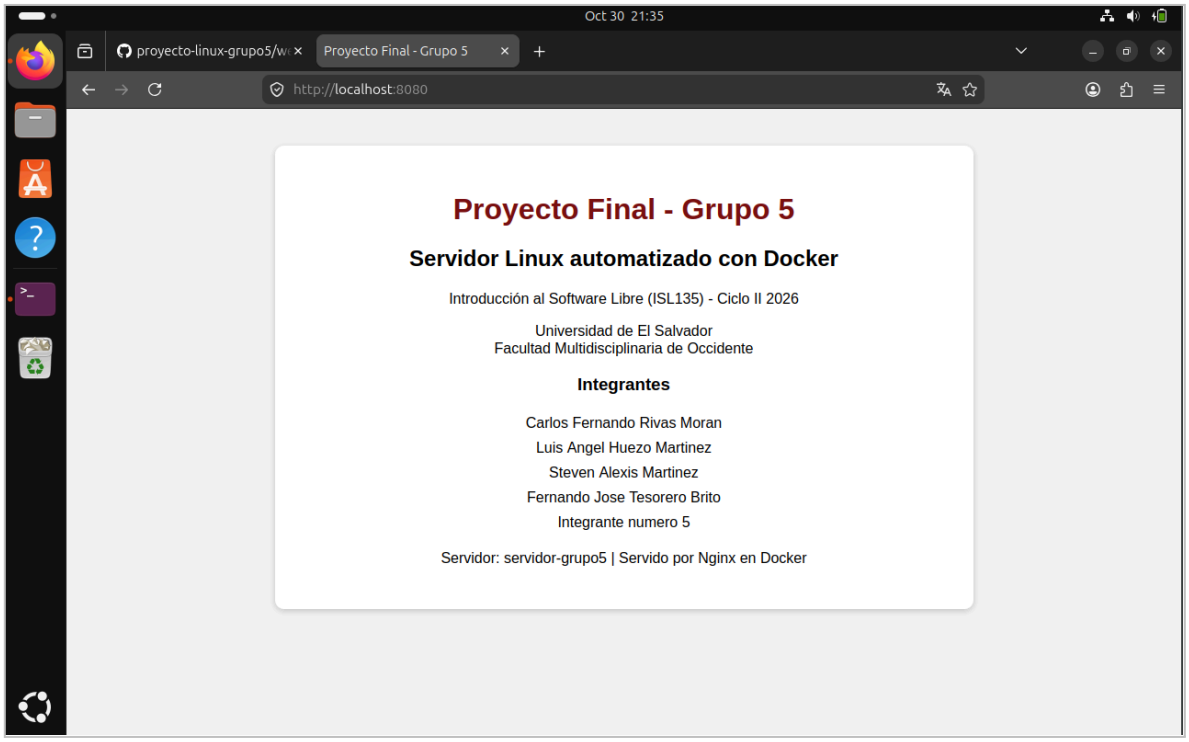


Figura 13. *Página del Grupo 5 servida por Nginx en http://localhost:8080.*

6. Docker avanzado

6.1 Dockerfile personalizado

Se creó el Dockerfile en /proyecto/ con la imagen base nginx:latest, la copia del contenido de web/ al directorio de Nginx, la exposición del puerto 80 y el comando de inicio de Nginx (la opción daemon off mantiene el proceso en primer plano para que el contenedor siga activo). Luego se construyó la imagen con el nombre servidor-grupo5.

```
FROM nginx:latest
COPY web/ /usr/share/nginx/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

cd /proyecto
docker build -t servidor-grupo5 .
docker images
```

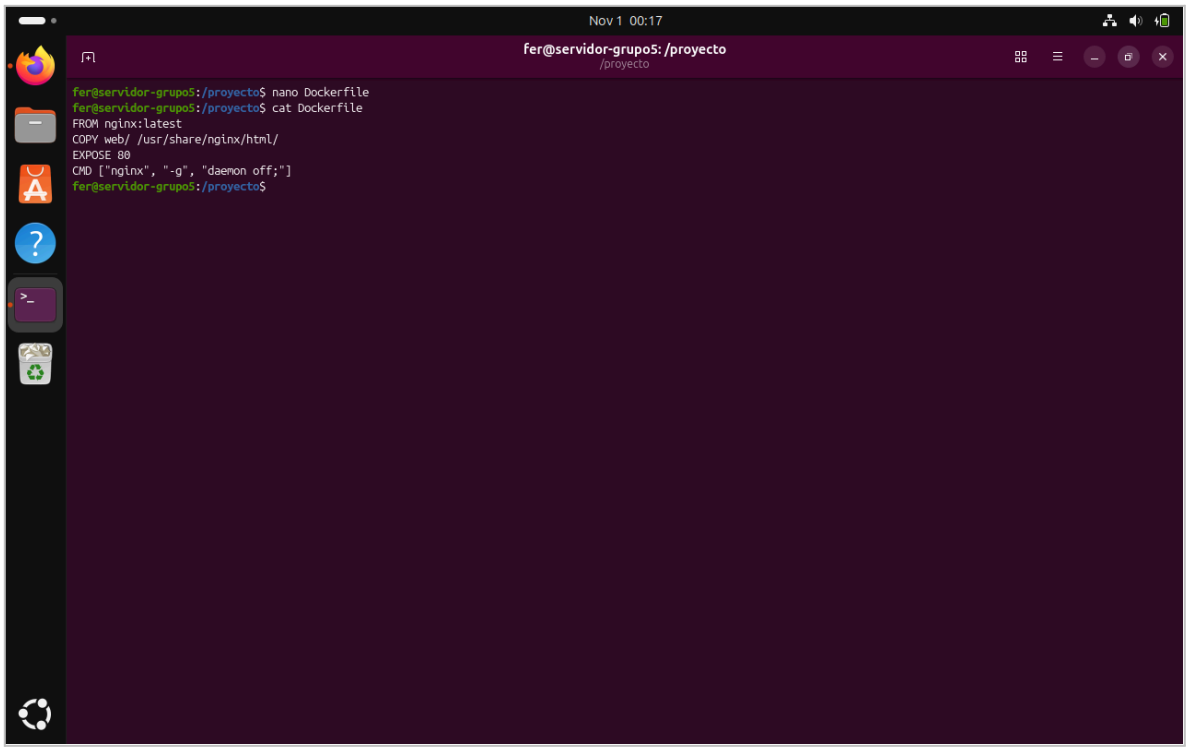


Figura 14. *Creación y contenido del Dockerfile.*

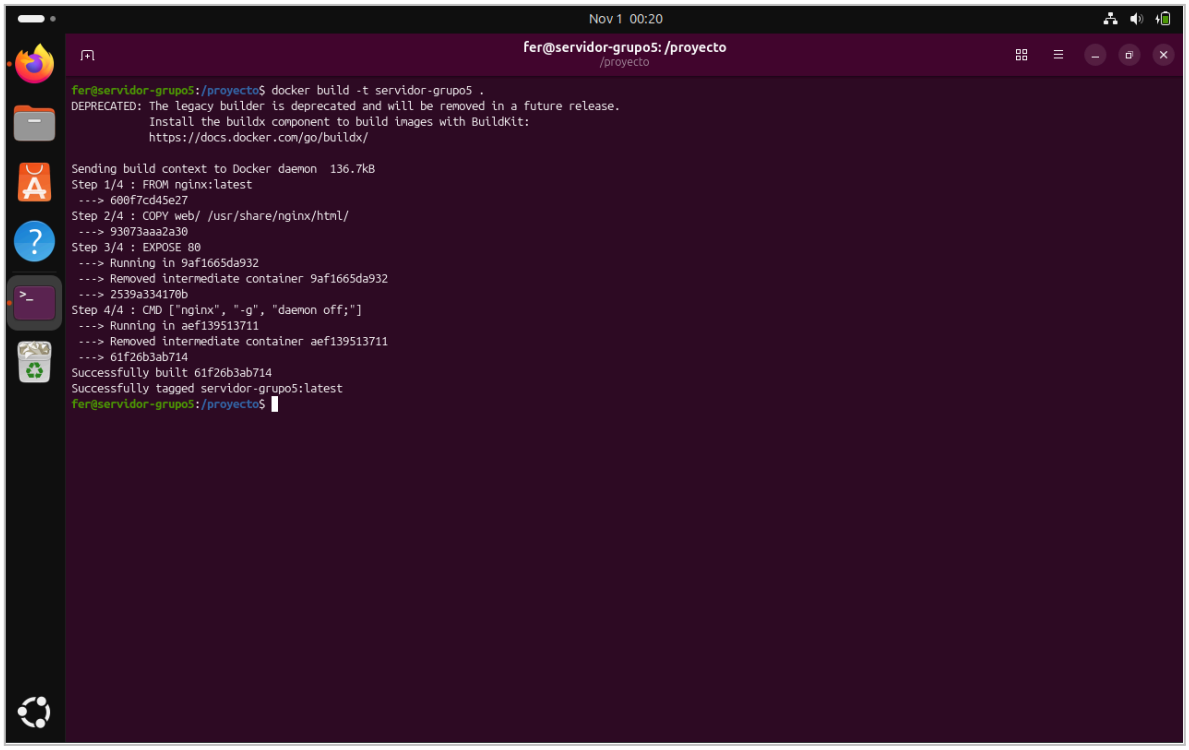


Figura 15. *Construcción de la imagen servidor-grupo5 con docker build.*

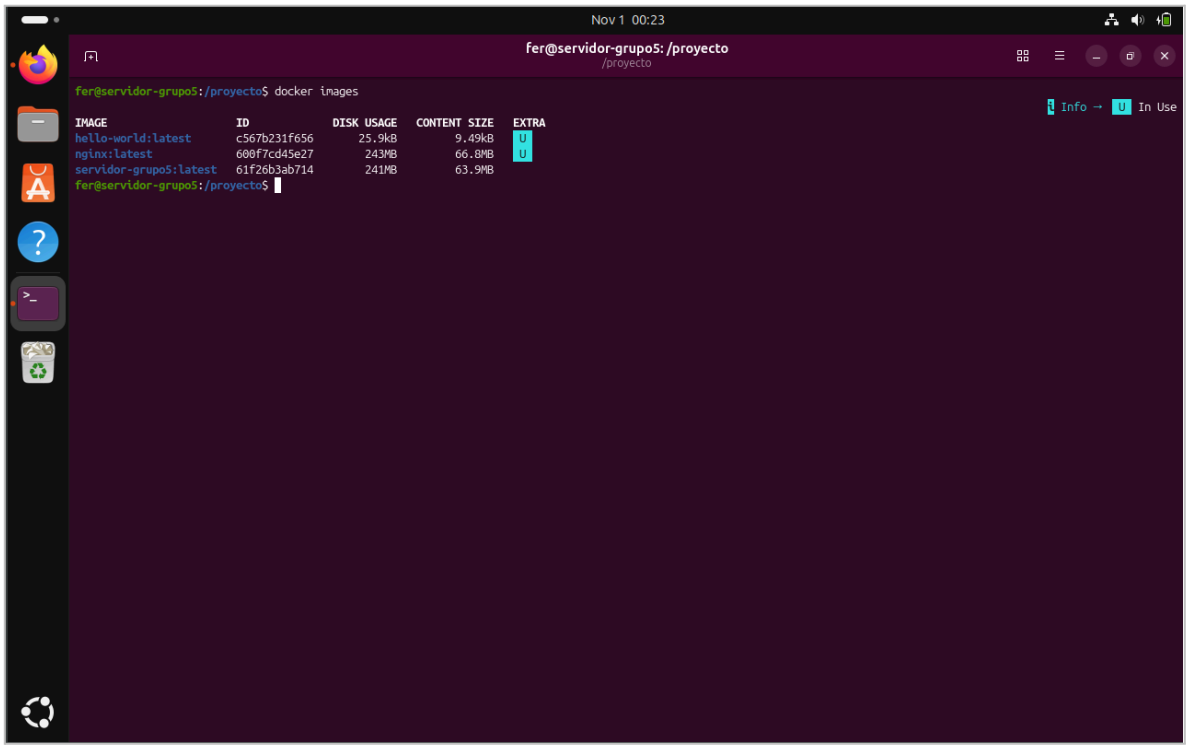


Figura 16. *docker images: la imagen servidor-grupo5 junto a nginx y hello-world.*

6.2 Contenedor con imagen personalizada

Se ejecutó un contenedor a partir de la imagen servidor-grupo5, mapeando el puerto 8081 del host al 80 del contenedor, y se verificó que el sitio funciona. Para compararlo con el Nginx oficial (puerto 8080) se modificó index.html: el contenedor oficial refleja el cambio de inmediato porque usa el directorio montado, mientras que la imagen personalizada conserva el contenido que se copió al construirla y solo cambia al reconstruir la imagen.

```
docker run -d --name servidor-grupo5 -p 8081:80 servidor-grupo5
docker ps
curl http://localhost:8081
```

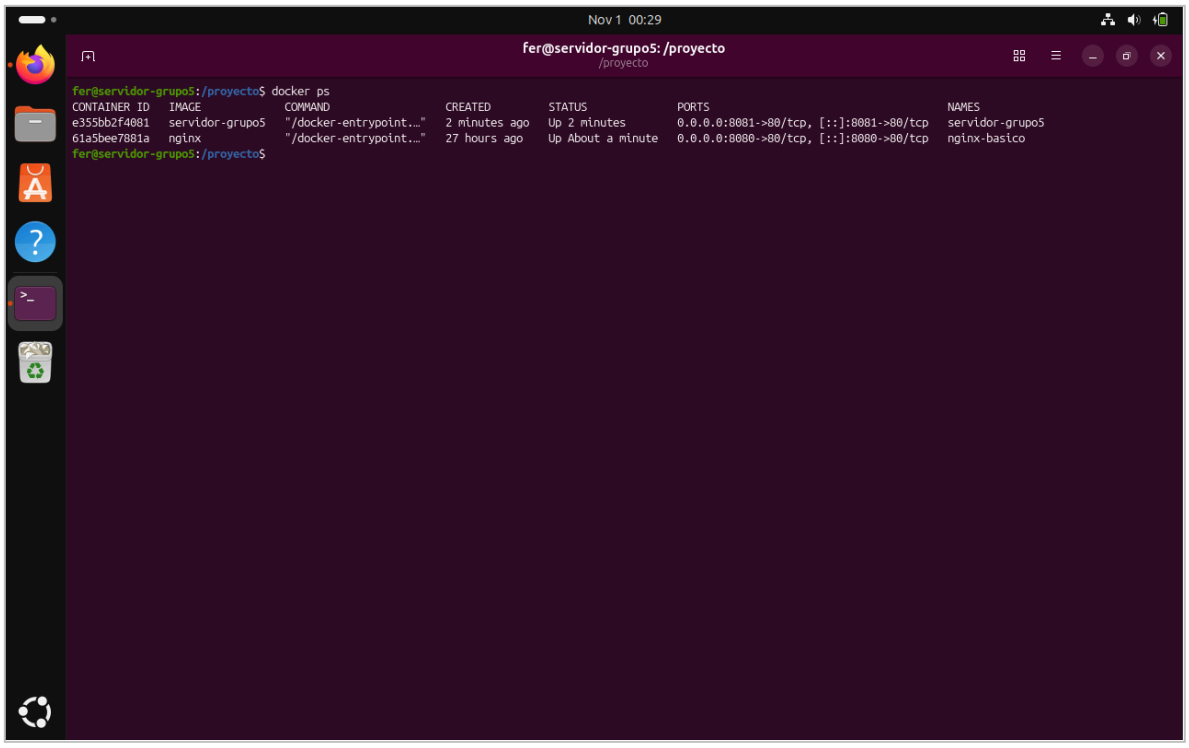


Figura 17. *docker ps: servidor-grupo5 en el puerto 8081 junto a nginx-basico en el 8080.*

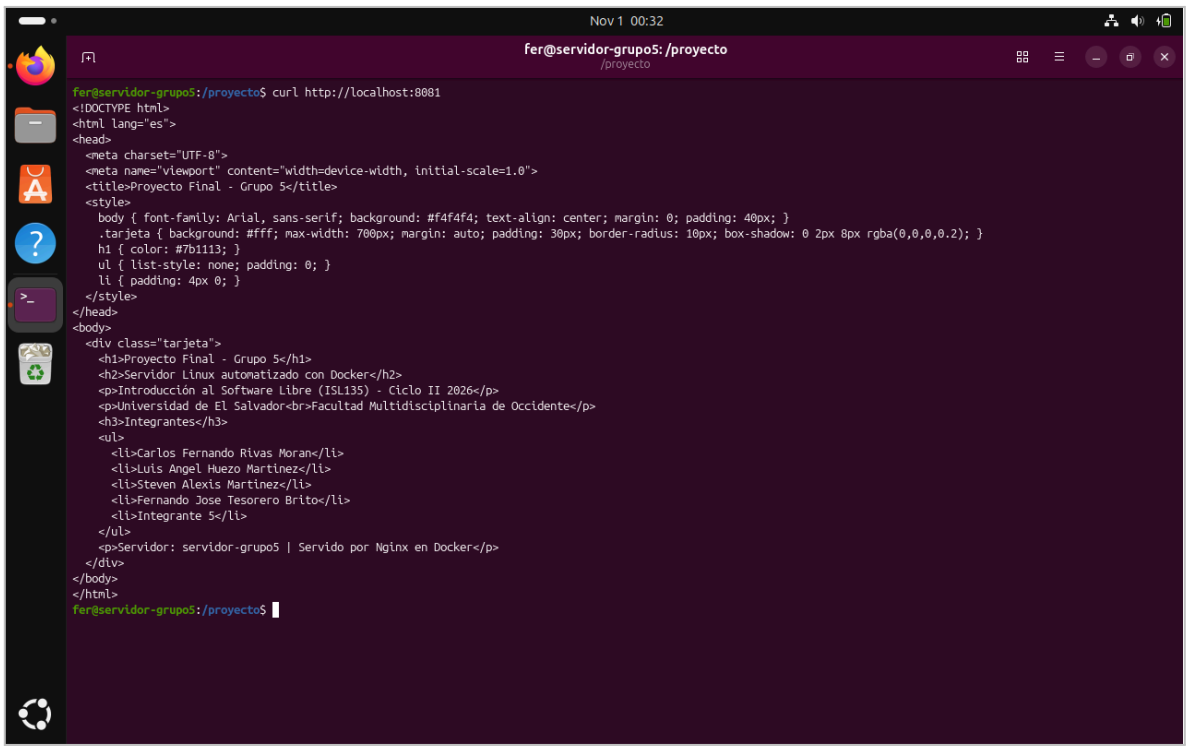


Figura 18. *curl a http://localhost:8081: el servidor responde con el sitio del grupo.*

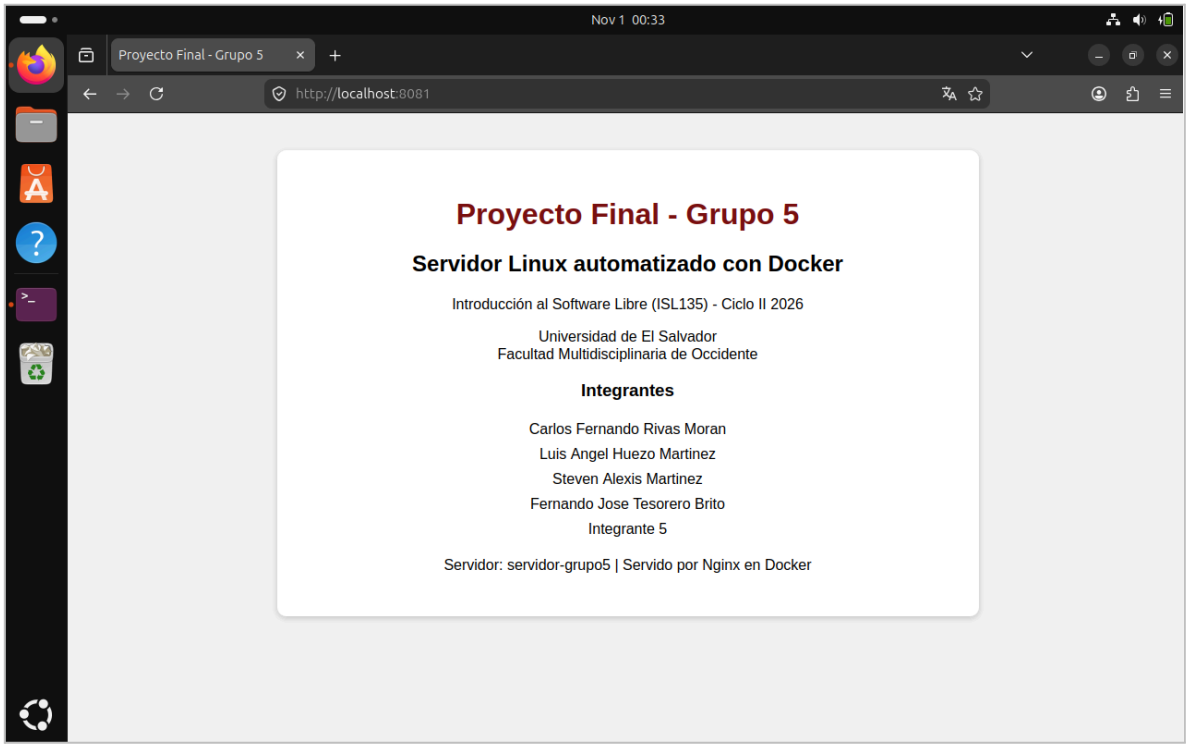


Figura 19. *Sitio servido por la imagen personalizada en http://localhost:8081.*

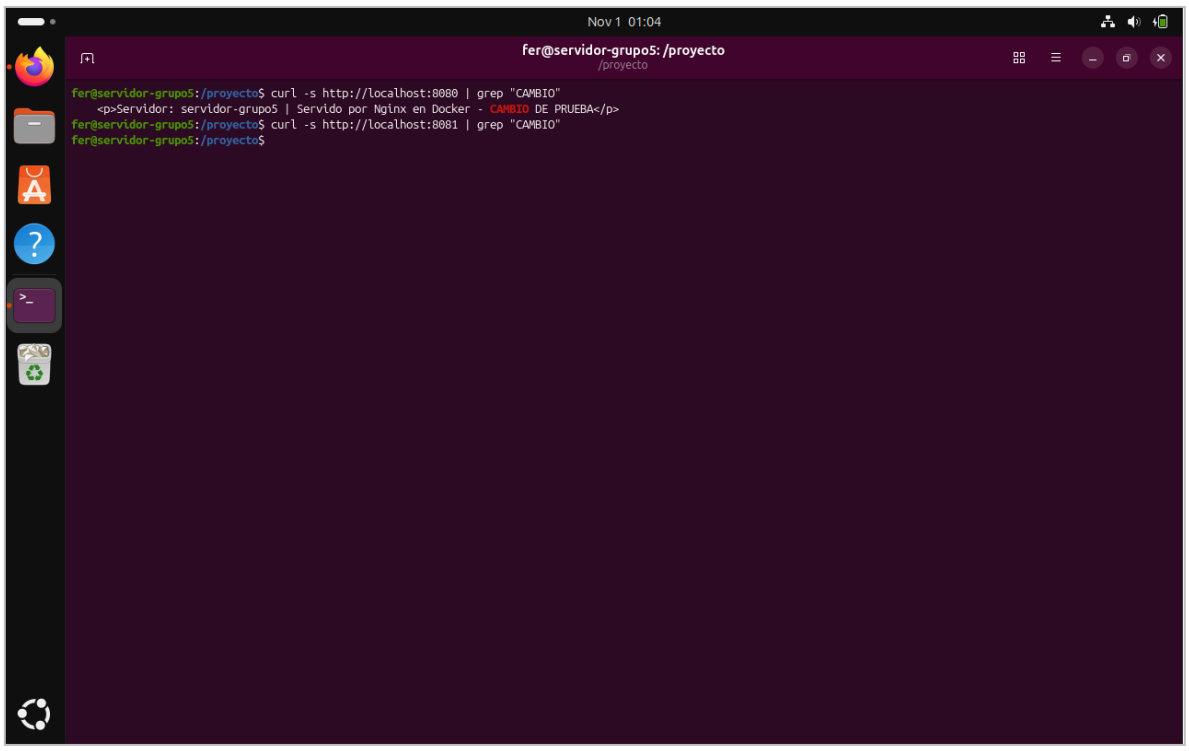


Figura 20. *Comparación: el cambio en index.html aparece en el puerto 8080 y no en el 8081.*

7. Redes y volúmenes Docker

7.1 Red personalizada

Se creó la red red-proyecto y se ejecutó un contenedor Nginx conectado a ella (nginx-red), mapeando el puerto 8082 del host. Se verificó la conexión a la red inspeccionándola y comprobando que el servicio responde.

```
docker network create red-proyecto
docker network ls
docker run -d --name nginx-red --network red-proyecto -p 8082:80 nginx
docker ps
docker network inspect red-proyecto
curl http://localhost:8082
```

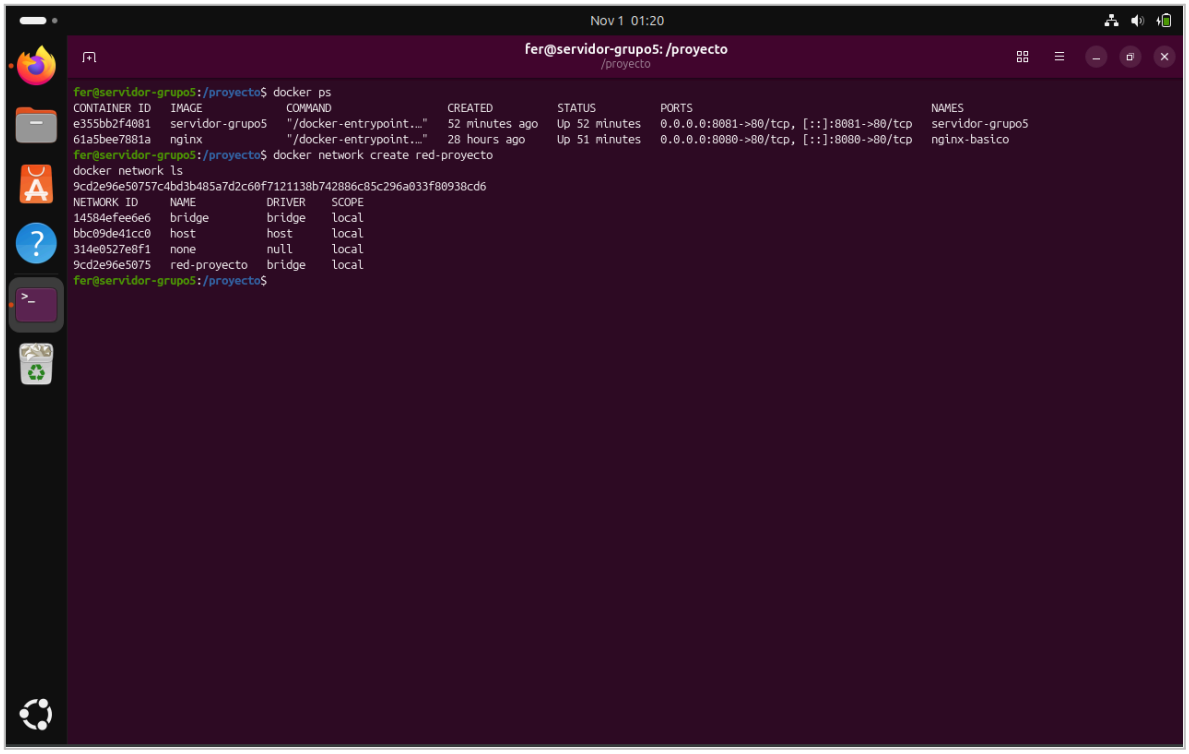


Figura 21. *Creación de la red red-proyecto.*

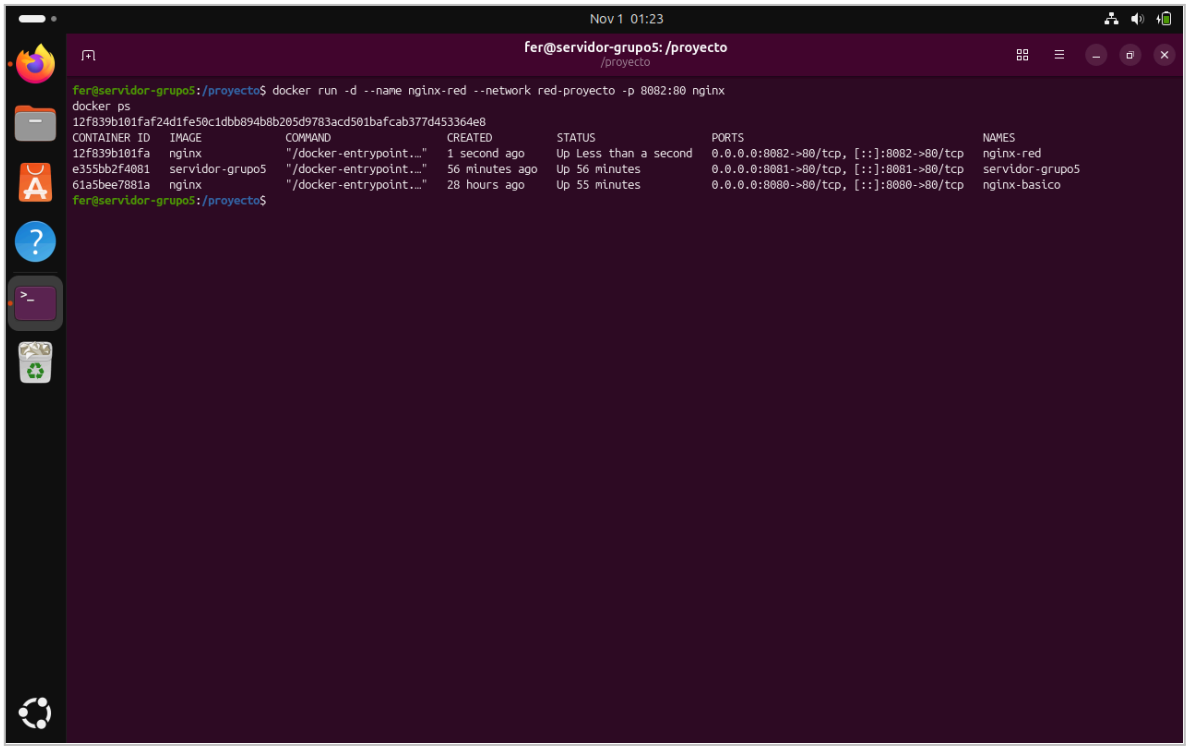


Figura 22. *Contenedor nginx-red conectado a red-proyecto en el puerto 8082.*

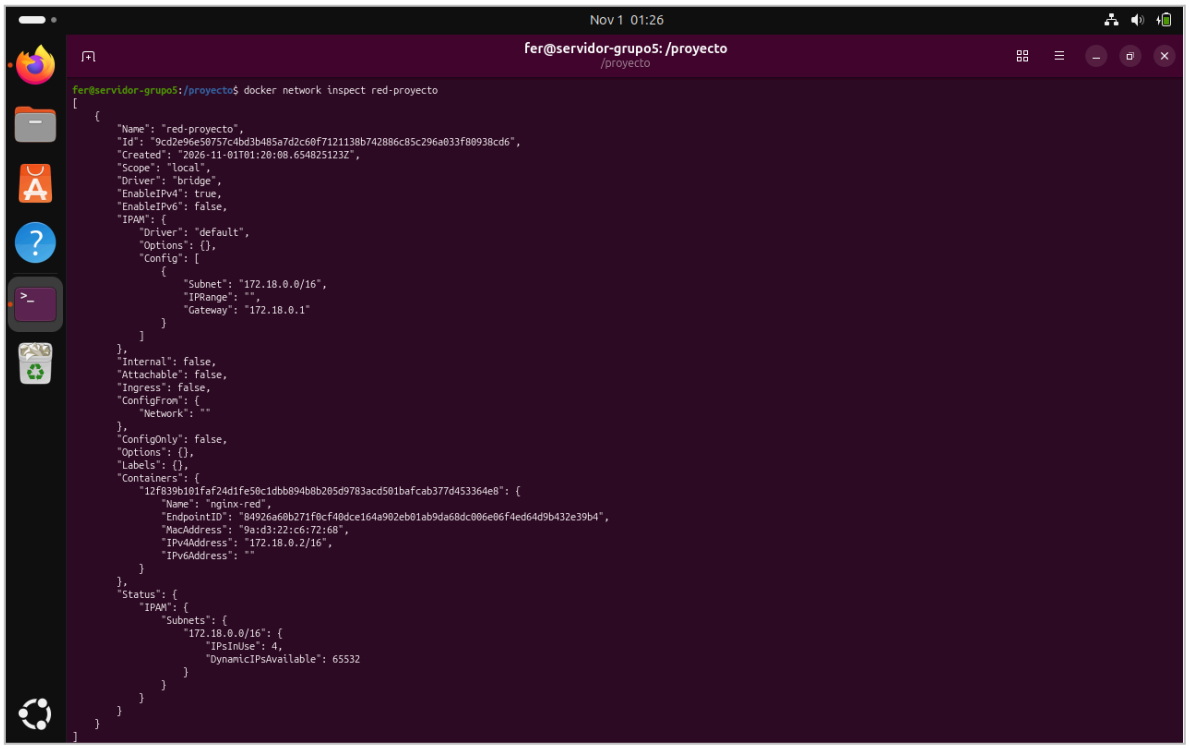


Figura 23. *docker network inspect red-proyecto.*

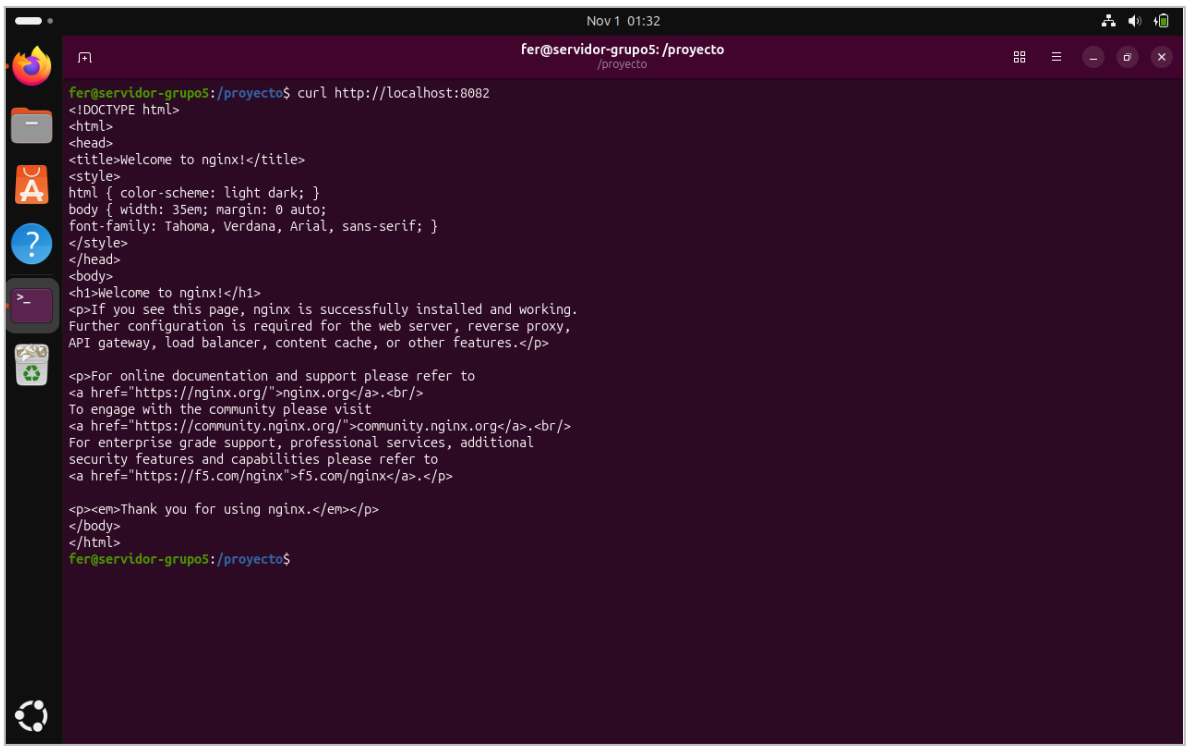


Figura 24. *curl a http://localhost:8082: Nginx responde.*

7.2 Volúmenes persistentes

Se creó el volumen volumen-web y se ejecutó un contenedor Nginx (nginx-volumen) que lo monta en el directorio de contenido web, con el puerto 8083 del host. Se verificó el montaje con docker volume inspect y docker inspect, y se probó la persistencia escribiendo un index.html dentro del volumen: el contenido se conserva aunque los contenedores se reinicien.

```
docker volume create volumen-web
docker volume ls
docker run -d --name nginx-volumen -p 8083:80 -v
volumen-web:/usr/share/nginx/html nginx
docker volume inspect volumen-web
docker inspect nginx-volumen | grep -A 12 'Mounts'
curl http://localhost:8083
```

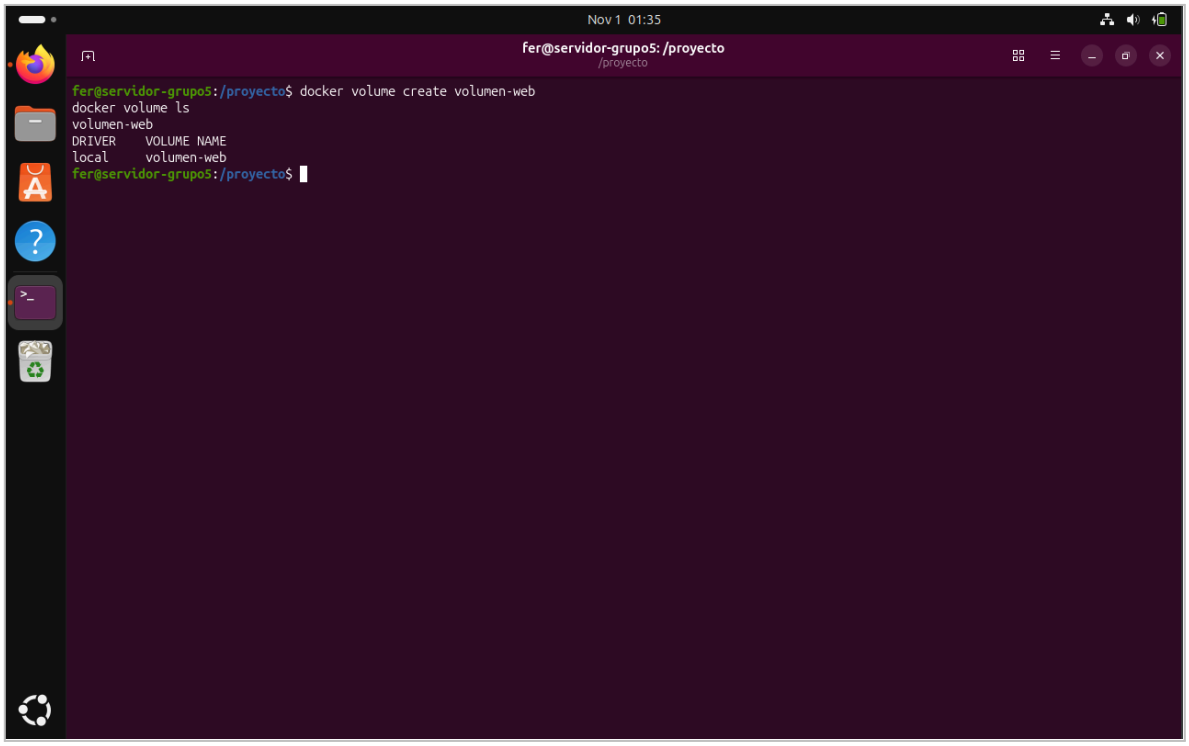


Figura 25. *Creación del volumen volumen-web.*

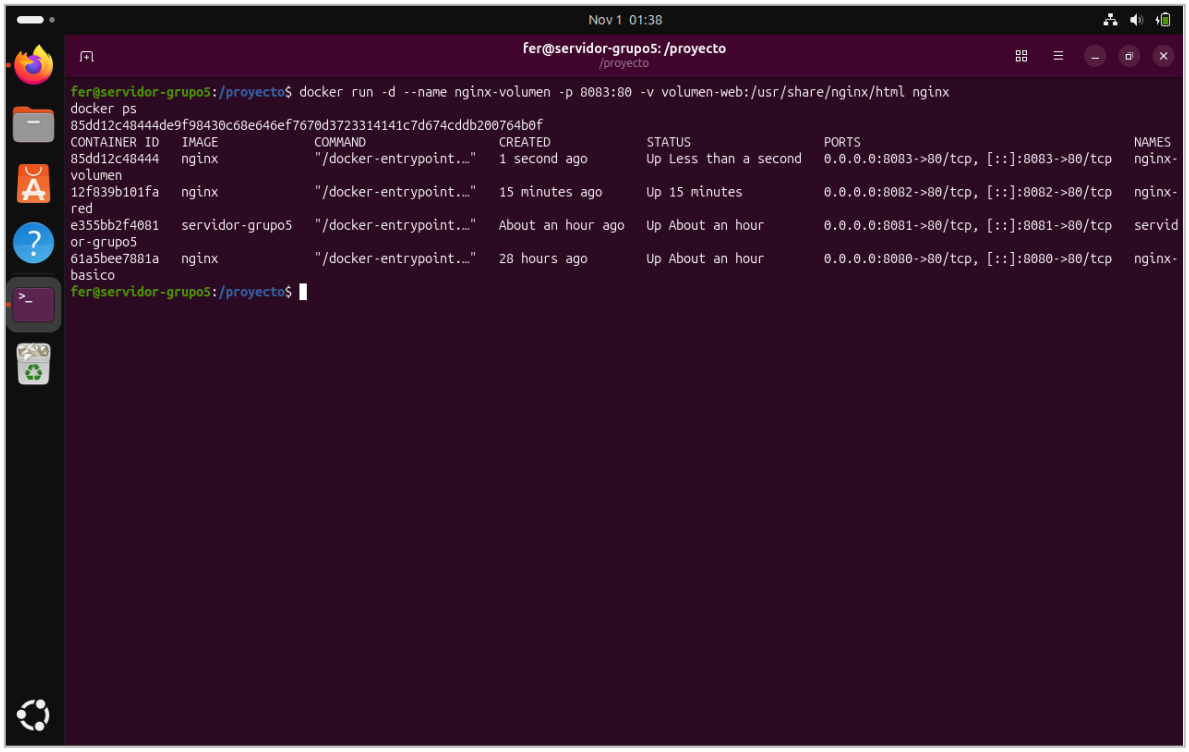


Figura 26. *Contenedor nginx-volumen con volumen-web montado, puerto 8083.*

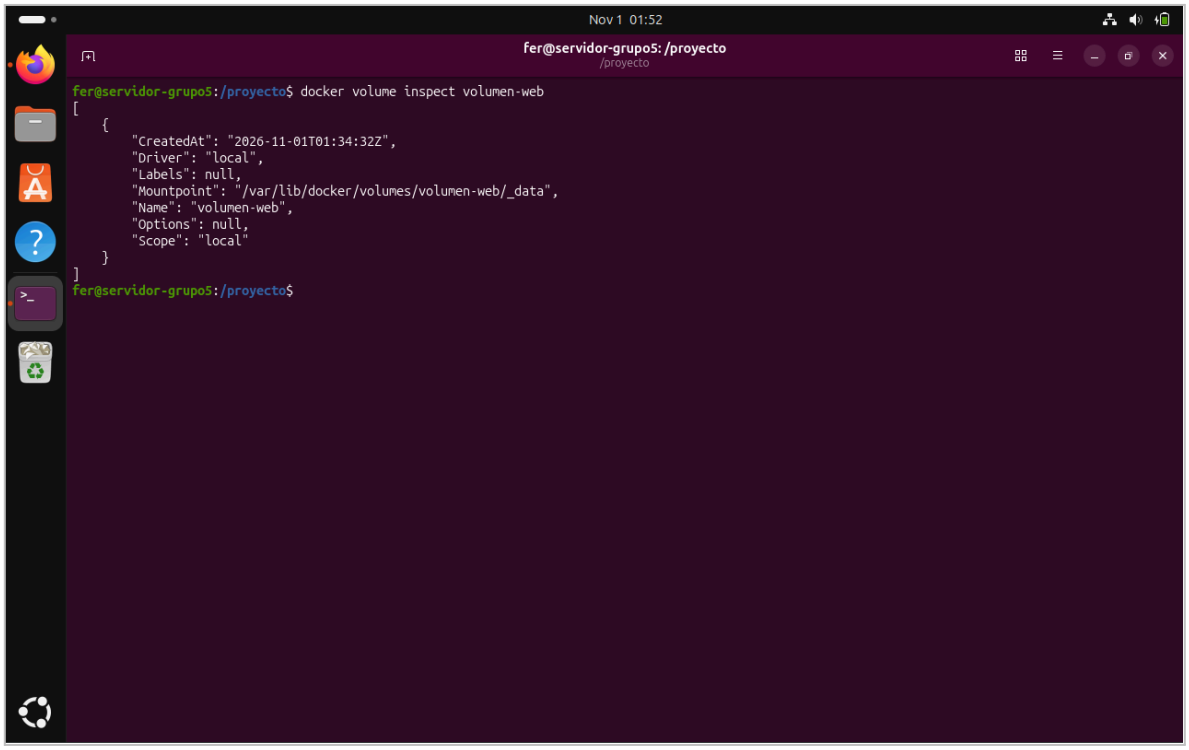


Figura 27. *docker volume inspect volumen-web.*

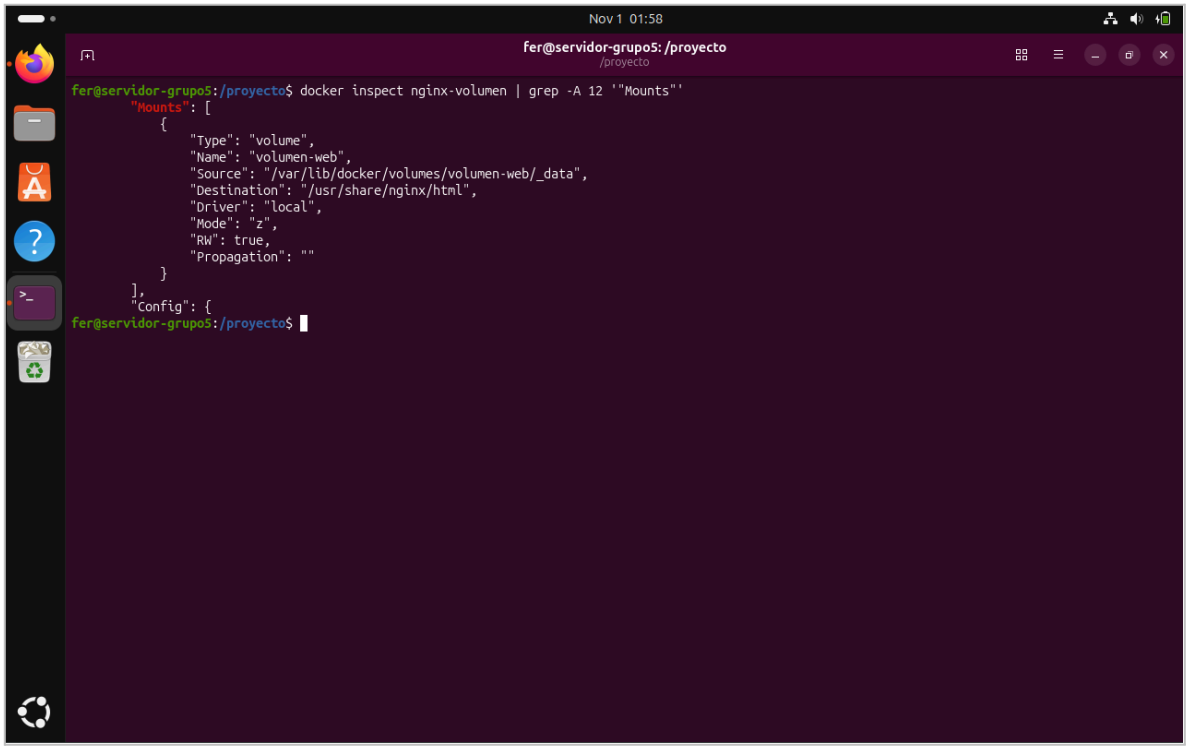


Figura 28. *docker inspect nginx-volumen: volumen-web montado en /usr/share/nginx/html.*

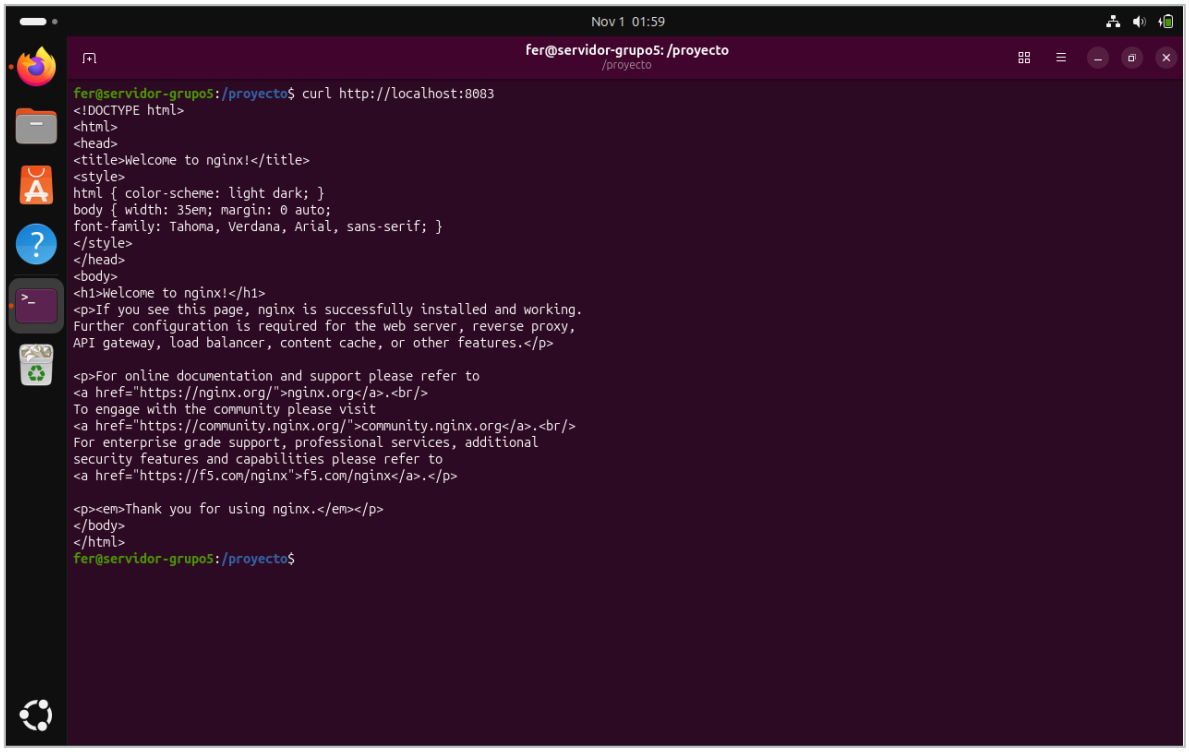


Figura 29. *curl a http://localhost:8083: página inicial de Nginx.*

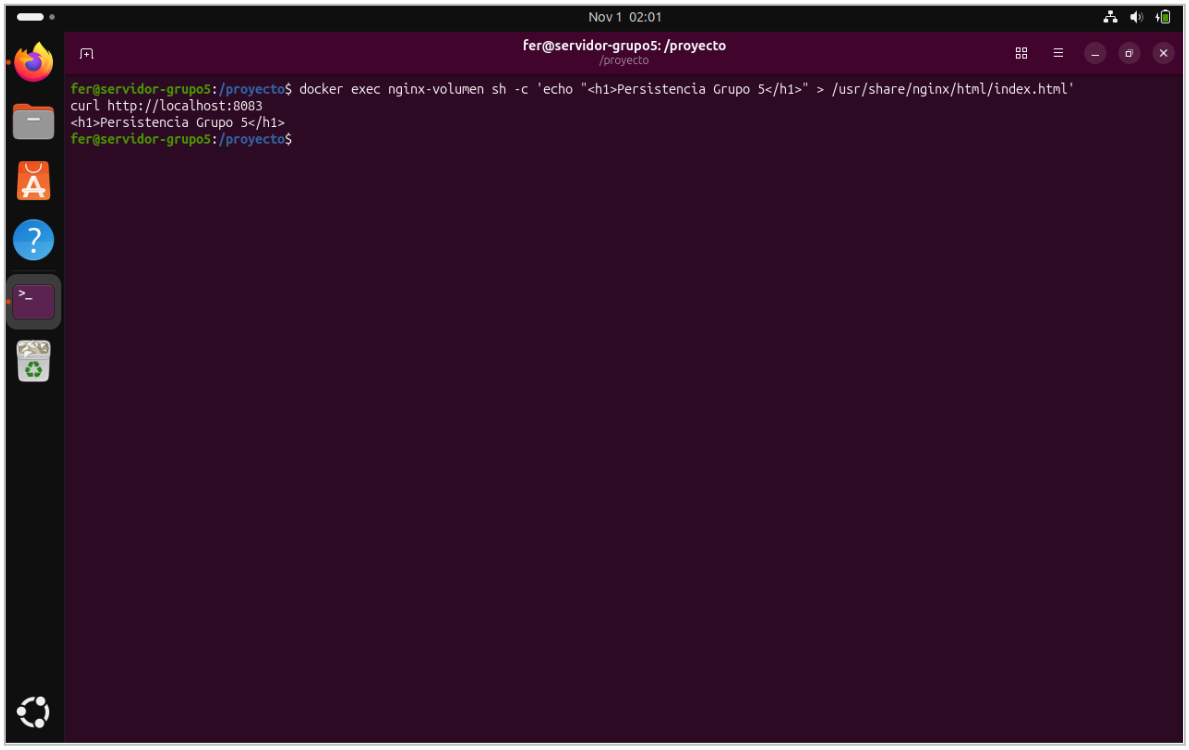


Figura 30. *Escritura de un index.html dentro del volumen con docker exec y comprobación con curl.*

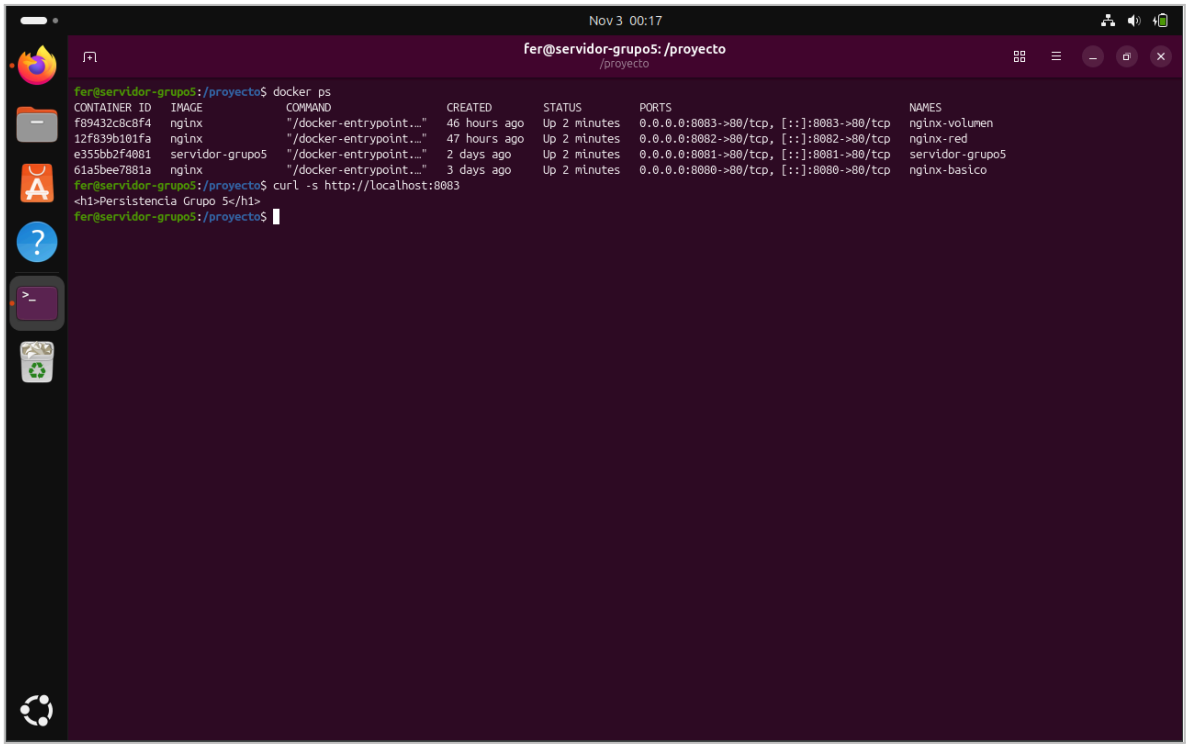


Figura 31. *Persistencia: tras reiniciarse los contenedores, el volumen conserva el contenido.*

8. Pruebas y verificación final

8.1 Pruebas de conectividad

Se ejecutó un contenedor Alpine Linux conectado a la red red-proyecto y se hizo ping al contenedor nginx-red por su nombre, comprobando la comunicación dentro de la red personalizada.

```
docker run -it --rm --name alpine-test --network red-proyecto alpine ping -c 4
nginx-red
```

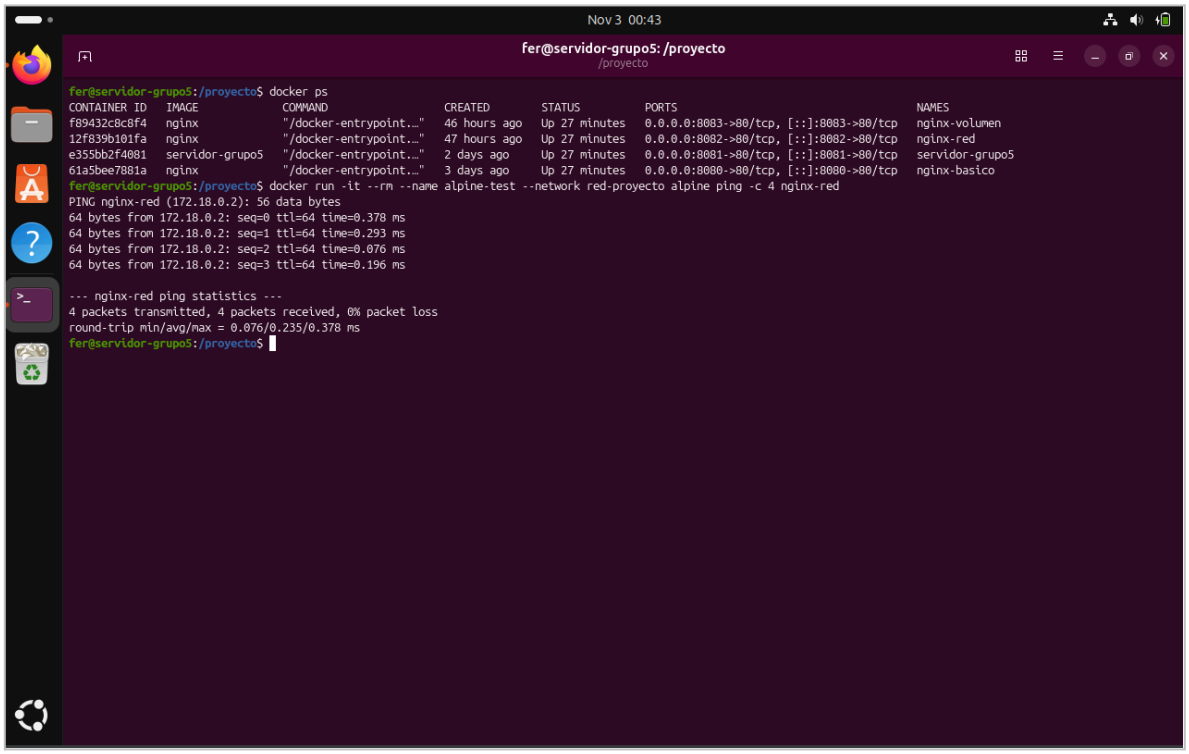


Figura 32. *Contenedor Alpine en red-proyecto: ping a nginx-red con respuesta.*

8.2 Verificación integral

Se comprobó que los cuatro servicios web responden en sus puertos (8080 a 8083), que todos los contenedores están en ejecución y que el script de monitoreo se ejecuta automáticamente cada 30 minutos (evidencia en el apartado 2.2). Por último se verifica que el repositorio Git esté actualizado con todo el proyecto.

```
curl -I http://localhost:8080
curl -I http://localhost:8081
curl -I http://localhost:8082
curl -I http://localhost:8083
docker ps
git status
git log --oneline
```

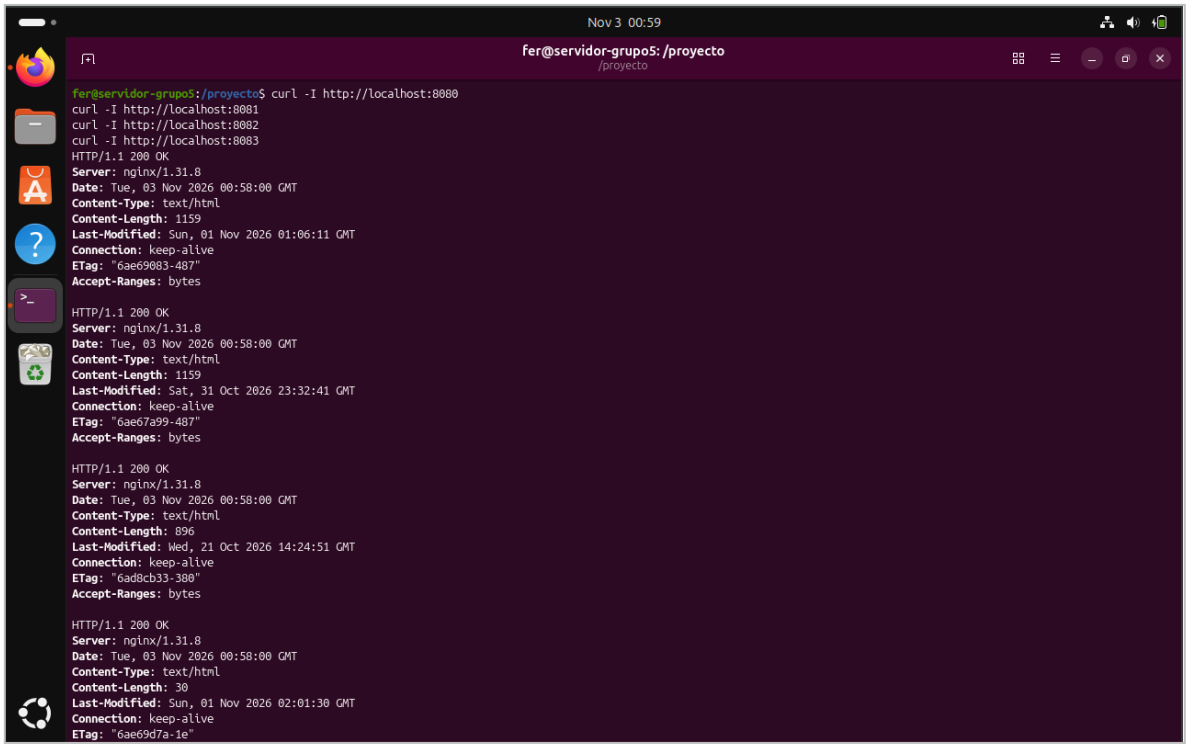


Figura 33. *curl -I a los puertos 8080, 8081, 8082 y 8083: HTTP 200 OK en los cuatro servicios.*

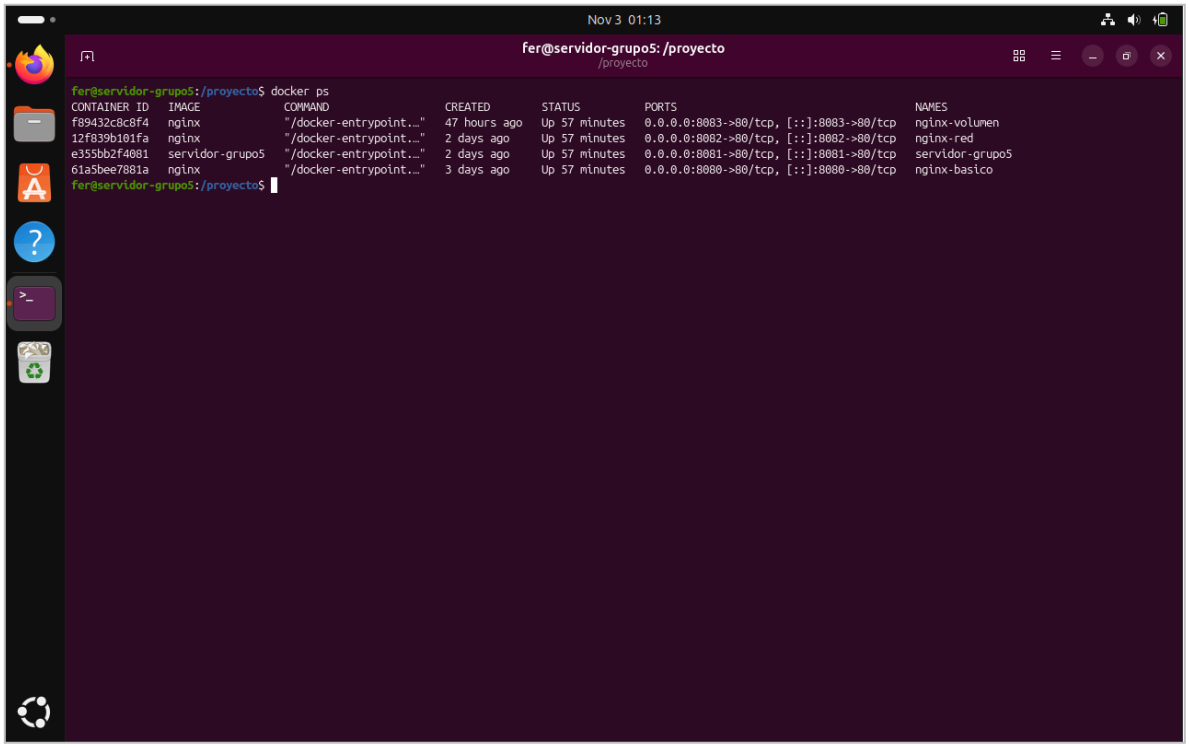


Figura 34. *docker ps: los cuatro contenedores en ejecución.*

Captura del repositorio actualizado

